

Starfighters—On the General Applicability of X-Wing

Deirdre Connolly¹, Kathrin Hövelmanns², Andreas Hülsing^{1,2}, Stavros Kousidis³, and
Matthias Meijers²

¹ SandboxAQ, USA

`durumcrustulum@gmail.com`

² Eindhoven University of Technology, The Netherlands

`kathrin@hoevelmanns.net`, `andreas@huelensing.net`, `research@mmeijers.com`

³ Federal Office for Information Security, Germany

`st.kousidis@gmail.com`

Abstract. In this work, we present a comprehensive analysis of QSF, the KEM combiner used by X-Wing (Communications in Cryptology 1(1), 2024). While the X-Wing paper focuses on the application of QSF to ML-KEM-768 and X25519, we discuss the combiner’s applicability to other post-quantum KEMs and ECDH instantiations. In particular, we establish the compatibility of QSF to KEMs based on variants of the Fujisaki-Okamoto transform by proving ciphertext second-preimage resistance (C2PRI) for these variants. Building on these results, we show that QSF is compatible with, to the best of our knowledge, all post-quantum KEMs currently standardized or considered for standardization—including ML-KEM, (e)FrodoKEM, HQC, Classic McEliece, and various NTRU variants. Notably, this means these schemes can be used with QSF to construct PQ/T hybrid KEMs.

In addition, we introduce QSI, a variant of QSF that combines two KEMs by hashing their shared keys, yielding a KEM that is IND-CCA-secure as long as one constituent KEM is IND-CCA-secure and the other is C2PRI-secure. We establish the same compatibility results for QSI as for QSF.

Finally, we analyze both QSF and QSI regarding (their preservation of) the recently introduced family of binding properties for KEMs.

Keywords: Post-quantum cryptography, KEM, KEM combiner, FO transform, hybrid, IND-CCA, C2PRI, HON-BIND, LEAK-BIND, MAL-BIND, QSF, X-Wing, QSI.

A.H. and M.M. were funded by an NWO VIDI grant (Project No. VI.Vidi.193.066). K.H. was supported by an NWO VENI grant (Project No. VI.Veni.222.397). Part of this work was carried out while M.M. was an intern at SandboxAQ.

Table of Contents

1	Introduction	2
2	Preliminaries	6
2.1	(Keyed) Hash Functions	7
2.2	Public-Key Encryption	8
2.3	Key Encapsulation Mechanisms	8
2.4	Fujisaki-Okamoto Transformation	10
2.5	KEM Combiners, QSF, and X-Wing	12
3	QSI: A KEM-Only Framework	13
3.1	Security: IND-CCA	16
3.2	Security: Binding	20
3.3	Plugging in FO-Based KEMs	27
4	QSF & QSI with Real-World KEMs	31
4.1	ML-KEM	31
4.2	HQC	32
4.3	(e)FrodoKEM	32
4.4	Classic McEliece	33
4.5	NTRU	33
4.6	Interpretation of Bounds	33
	References	37
A	Definitions	37
A.1	Hash Functions	37
A.2	Key Encapsulation Mechanisms	39
A.3	QSF	40
B	Hash Input Size and Performance Impact	40
C	QSI: Variant and Multi-User/Challenge Security	41
C.1	Variant: Additionally Hashing an Encapsulation	41
C.2	Multi-User/Challenge IND-CCA	43
D	Binding: Remaining Definitions and Security Proofs	44
D.1	Definition: Malicious Binding (MAL)	44
D.2	Honest Binding (HON) for QSF & QSI	45
D.3	Malicious Binding (MAL) for QSF & QSI	49

1 Introduction

The store-now-decrypt-later attack—where an attacker *stores* encrypted data *now* to *decrypt* it *later* when it gains access to a sufficiently powerful quantum computer—puts a certain urgency on the transition to post-quantum cryptography, i.e., cryptography that is resistant to attacks from both classical and quantum computers. Consequently, security agencies worldwide are urging institutions and companies to initiate this transition [Can25, Com25, Nat25b, Nat25a]. While exact timelines differ, all of them target completion within the next decade. Some prominent security advisories recommend hybrid approaches, often referred to as *PQ/T hybrids*, which combine a post-quantum solution with a traditional one [Com25]. This recommendation stems from the commonly noted concern that the to-be-deployed post-quantum schemes have not yet undergone the same level of scrutiny—both in terms of cryptanalysis and implementation security—as traditional schemes based

on, e.g., elliptic curves or RSA. Hybrids aim to minimize risk in case the post-quantum solution proves vulnerable or falters. Typically, these hybrids are defined using *combiners*: cryptographic constructions that take several schemes as input and produce a single scheme that is secure as long as *at least one* of the input schemes is secure.

When considering quantum computers, the core primitive for achieving confidentiality in the public-key setting is the key encapsulation mechanism (KEM). Unlike conventional public-key encryption (PKE) schemes, KEMs are specifically designed for the transport of secret key material—often referred to as shared key or session key, after its typical use case. Instead of transmitting the encryption of a message via a ciphertext, a KEM transmits the encryption of key material via an *encapsulation* (sometimes also called ciphertext). The encapsulation algorithm of a KEM takes only the recipient’s public key pk as input—particularly, it does *not* take a message as input, unlike the encryption algorithm of a PKE scheme—and produces both a shared key k and an encapsulation c . The recipient can then *decapsulate* c using the private key corresponding to pk , recovering k . Because the encapsulation algorithm generates the shared key, it is easier to achieve strong security guarantees with KEMs than it is with traditional PKE schemes.

In the context of KEMs, the goal of a combiner is typically to combine multiple KEMs such that the resulting KEM satisfies the conventional *indistinguishability under chosen-ciphertext attacks* (IND-CCA) property, provided *at least one* of its constituent KEMs does. (However, some combiners may require additional security properties from the KEMs that do *not* satisfy IND-CCA, as discussed below.) The theoretical problem of constructing such combiners was thoroughly studied in a detailed work by Giacon, Heuer, and Poettering [GHP18]. In particular, they analyzed several methods of combining shared key/encapsulation pairs (k_0, c_0) and (k_1, c_1) , generated by two (constituent) KEMs, into a final shared key/encapsulation pair produced by the combiner. While virtually all of their proposals construct the final encapsulation as (c_0, c_1) , each differs in how they produce the final shared key, which may depend on the shared keys, encapsulations and public keys of the constituent KEMs. For example, they observe that producing the final shared key as

$$k = H(k_0, k_1, c_0, c_1)$$

results in an IND-CCA-secure combiner if H is a so-called split-key pseudo-random function family, which they further show is the case when H is modeled as a random oracle (see Section 2.1 for some further details).

Beyond the black-box constructions presented in [GHP18], there has been significant interest in optimizing combiners for high-throughput applications, such as TLS. In this context, Barbosa, Connolly, Diogo Duarte, Kaiser, Schwabe, Varner, and Westerbaan put forth X-Wing in [BCD⁺24], arguing that the above combiner can be optimized when using ML-KEM-768. Particularly, they demonstrated that ML-KEM-768’s encapsulation may be omitted from the input to H because this KEM satisfies ciphertext second-preimage resistance C2PRI, a novel security notion introduced by the authors. For their argument, however, it is crucial that ML-KEM-768 satisfies this notion even if its underlying (lattice-based) computational hardness assumption proves to be incorrect. Furthermore, rather than using another IND-CCA-secure KEM for the other scheme in the combiner, X-Wing employs X25519, a non-interactive key exchange (NIKE) that only achieves the (weaker) one-wayness security notion OW-PCVA.⁴ To compensate for this, X-Wing includes the X25519 encapsulation in the input to H , ensuring (pre-quantum) IND-CCA security of X-Wing in case ML-KEM-768 is found

⁴ Although a NIKE can trivially be transformed into a KEM by performing an ephemeral-static key exchange—letting the ephemeral public key and (final) derived key serve as the encapsulation and shared key, respectively—this does *not* result in an IND-CCA-secure KEM for X25519 due to its algebraic properties. This can be fixed by instead establishing the shared key as a hash of the derived key and the ephemeral public key [CS03].

to be vulnerable. For resilience against multi-user attacks, X-Wing additionally incorporates the X25519 public key in the input to H . In summary, X-Wing computes its final shared secret as

$$k = H(k_0, k_1, c_1, \text{pk}_1, \text{context}) ,$$

where k_0 denotes ML-KEM’s shared key, and k_1 , c_1 , and pk_1 denote X25519’s shared key, encapsulation and public key.

Afterward, Josefsson noted that the artifacts processed by the general combiner from [GHP18] can be quite large, which may pose challenges for, e.g., constrained devices [Jos25]. To address this issue, Josefsson proposed Chempat, a KEM combiner that allows for memory reduction by hashing these large artifacts (when possible) before using them as inputs to H :⁵

$$k = H(k_0, k_1, H(c_0, c_1), H(\text{pk}_0, \text{pk}_1), \text{context}) .$$

Several other proposals have emerged over time, such as ETSI’s CasKDF and CatKDF [ETS25], and discussions regarding the choice of combiner have become increasingly intense. In part, this is because the original analysis from [GHP18] does not apply to the newer combiners. For instance, the work introducing X-Wing also describes the underlying generic combiner, QSF. However, QSF imposes additional requirements on both of its constituents, as discussed above (for X-Wing). While the authors of X-Wing did note that the C2PRI property holds for the variant of the Fujisaki-Okamoto (FO) transform used by ML-KEM, their detailed analysis was limited to the specific schemes used for X-Wing, ML-KEM-768 and X25519. Moreover, recent work by Cremers, Dax, and Medinger [CDM24] has sparked further debate, as the novel security properties they introduce—called binding properties—are used as arguments both for and against certain combiners.

The GHP, Chempat, QSF, and X-Wing constructions have been picked up by standardization bodies, including IETF/IRTF [CBG25, CB25]. However, several questions remain unanswered. The primary advantage of QSF and X-Wing is their efficiency, while that of GHP and Chempat is their versatility—in the sense that they can be used with any two KEMs and provide IND-CCA security if at least one of the constituent KEMs is IND-CCA-secure.⁶ In contrast, for QSF, it is unclear whether ML-KEM-768 can even be replaced by a different KEM without sacrificing security guarantees in case that the hardness assumption of this post-quantum KEM fails. This is an issue as there are KEMs *other* than ML-KEM, such as HQC, Classic McEliece, and FrodoKEM, that are considered in practice and for standardization (by, e.g., ISO). Moreover, to the best of our knowledge, there has been no formal analysis of the different proposals regarding their impact on binding properties prior to this work. Since it is trivial to devise a combiner that undermines any binding properties of its constituents, performing such an analysis seems crucial if these properties are a security concern.

OUR CONTRIBUTION. In this work, we take an extensive look at the QSF proposal and its underlying ideas to fill the previously mentioned gaps for QSF. Specifically, we first introduce and analyze a variant of QSF that combines two arbitrary KEMs (encompassing the case of PQ/T hybrids), so that it is IND-CCA secure as long as at least one constituent KEM satisfies IND-CCA and the other satisfies C2PRI. We call this variant *Quantum Superiority Interceptor* (QSI), which computes the final shared key as

$$k = H(k_0, k_1, \text{context}) .$$

We prove that this fulfills the above claim of preserving IND-CCA, using C2PRI. In particular, for PQ/T hybrids this means that, if the post-quantum KEM’s IND-CCA security fails, QSI remains

⁵ In addition, Chempat makes several concrete choices concerning, e.g., combinations of algorithms; however, this is not relevant to this work.

⁶ Although there is no detailed proof for Chempat, Bernstein has rightfully pointed out that the additional hashing steps do not make a difference in case H is modeled as a random oracle.

IND-CCA-secure as long as the post-quantum KEM still satisfies C2PRI and the pre-quantum KEM satisfies IND-CCA. Similarly, if the pre-quantum KEM’s IND-CCA security fails (e.g., under attack by a quantum adversary), QSI remains IND-CCA-secure as long as the pre-quantum KEM still satisfies C2PRI-secure and the post-quantum KEM satisfies IND-CCA.

In essence, QSI is a modification of QSF that optimizes the inputs to the combiner hash when combining two KEMs that were designed to be IND-CCA and C2PRI. For completeness, we also demonstrate that one can remain conceptually close to QSF for combining two KEMs. That is, by additionally passing in the encapsulation of KEM_1 , we show the resulting (combiner) KEM is IND-CCA-secure whenever either (1) KEM_0 is IND-CCA-secure or (2) KEM_0 satisfies C2PRI and KEM_1 satisfies OW-PCVA, a one-way security notion slightly weaker than IND-CCA.⁷

Beyond IND-CCA, we investigate the binding properties of QSF and QSI. As mentioned before, a combiner does not necessarily satisfy or preserve any binding properties—indeed, it may even fail to retain those that *are* satisfied by its components. For QSI, we demonstrate that it preserves any binding properties held by its constituent KEMs. However, the “nature” of this preservation depends on the specific property: in some cases, QSI satisfies the property if *at least one* of its constituent KEMs does (we refer to this as *existential* preservation) while in other cases, it requires *both* constituent KEMs to satisfy the property (we refer to this as *universal* preservation). For QSF, the analysis yields more diverse results. Depending on the property, QSF may (1) inherently satisfy the property, i.e., without relying on any properties of its constituents; (2) preserve the property of its sole constituent KEM; or (3) fail to satisfy the property altogether. This somewhat stark difference with QSI primarily stems from the fact that QSF incorporates all artifacts from its constituent NIKE into final key derivation, which effectively removes any dependency on binding-like properties of the NIKE, simplifying parts of the analysis and leading to different results.

Furthermore, we examine the general applicability of QSF and QSI. For QSF, the authors have shown that the NIKE component can be instantiated using any nominal group, immediately covering all relevant elliptic-curve Diffie-Hellman constructions (see Remark 13). Therefore, our focus primarily shifts to analyzing when a KEM achieves C2PRI. Since nearly all post-quantum KEMs utilize (a variant of the) FO transform [FO13, HHK17], we discuss the conditions under which FO-based KEMs satisfy C2PRI. This analysis proved to be somewhat challenging, as almost no two schemes employ the same variant of FO. Ultimately, we show that C2PRI holds for all variants of the FO transform originally discussed in [HHK17], their extensions with key confirmation [BP18], and the more recently introduced salted versions [ABD⁺23, GHS25]. The most common variants of FO employ a mechanism called *implicit rejection*, which essentially means that decapsulation rejects ciphertexts by returning a pseudo-random value (rather than an explicit failure symbol). For these variants, we additionally show that C2PRI is achieved regardless of whether the same function is used to compute both the session key and the rejection value.

Lastly, we apply these results to all post-quantum KEMs currently considered for standardization—including ML-KEM-1024, HQC, (e)FrodoKEM, Classic McEliece, NTRU Prime, NTRU-HPS, and NTRU-HRSS—demonstrating that any of them can be used with QSF and QSI to construct a PQ/T hybrid KEM.

PURPOSE. This work is not intended to take a stance for or against combiners that additionally require C2PRI from one of their constituent KEMs—there are many arguments on both sides of the debate. Instead, our goal is to formally prove the guarantees obtained when using QSI (or QSF) and to specify the necessary assumptions.

On the one hand, our results indicate that requiring C2PRI will be unproblematic in many relevant cases: the KEMs considered here already achieve this property through a secure hash function. Hence, requiring C2PRI from the constituents does not seem much of an additional assumption. Further,

⁷ OW-PCVA is essentially an abstraction of the strong Diffie-Hellman assumption for KEMs.

QSI itself already relies on the security of a hash function, and a compromise of that hash function would nullify the combiner’s security in any case.⁸ To limit the attack surface, it therefore seems reasonable to reuse that same hash function for both constituent KEMs and QSI, provided that proper domain separation is employed for different applications.⁹

On the other hand, a specific KEM implementation may be rendered insecure due to implementation flaws. Such flaws could invalidate the KEM’s C2PRI and, in turn, the guarantees of the combiner. This raises, for example, the question of whether the combiner should be used as a measure against implementation-level failures.

In the end, whether reliance on C2PRI is reasonable remains a case-dependent judgment. We hope this work helps make that judgment on solid, formal grounds.

CONCURRENT WORK. After a preprint of this work appeared, two independent and concurrent works [ABK25, KSW25b] were published, each reproducing parts of our result. In [ABK25], the authors analyze the construction we introduce as QSI and prove the same IND-CCA bound. They also study the C2PRI property for several FO-based KEMs. Their proofs differ considerably from ours, with their final bound being slightly less tight having an additional square-root over the entire expression. This difference arises from the distinct techniques used to bound the advantage of quantum adversaries. Nevertheless, both results yield the same bit security. While the authors do not address binding properties nor provide exact C2PRI bounds for various KEMs, they do show that the QSI approach generalizes to more than two KEMs and discuss alternative instantiations of the function used for the final key derivation, beyond hash functions modeled as random oracles.

In [KSW25b], the authors analyze the binding properties of combiners for two constituent KEMs using a GHP-like framework. That is, the construction they analyze constructs its public keys, secret keys, and encapsulations by concatenating those of the constituents, but derives its shared key by applying a (generic) function to the constituents’ shared keys and, optionally, public keys and encapsulation. Different instantiations of this function then correspond to the concrete combiners discussed earlier. When instantiated to match QSI, their binding results closely align with ours with only minimal differences. They additionally derive an alternative bound for a specific encapsulation-binding property, based on different binding properties from the constituent KEMs, which we do not consider.

Thus, while there is overlap with our work, each concurrent work addresses isolated parts of the overall picture. Our contribution lies in a unified, coherent, practice-oriented framework. More specifically, we (1) give a generic analysis of the security properties relevant in practice and standardization for the combiners QSF and QSI; (2) provide a systematic C2PRI analysis for the FO transforms underlying practically deployed post-quantum KEMs; and (3) instantiate these results for concrete schemes, obtaining explicit bounds and coverage of essentially all practically relevant FO-based KEMs.

2 Preliminaries

Before proceeding, we review the relevant concepts.

⁸ Technically, the hash function property required for C2PRI is different than the one needed by the combiner; however, they are comparable in “strength”.

⁹ Additionally, if constructive redundancy for the hash is desired, *hash combiners* can provide this for different properties (c.f. [Her05, FLP14]).

2.1 (Keyed) Hash Functions

For the purposes of this work, a hash function is a function $H : \mathcal{X} \rightarrow \mathcal{Y}$ where $\mathcal{X}$ and $\mathcal{Y}$ are referred to as input (or message) space and output (or digest) space, respectively. A keyed hash function (KHF), then, adds an explicit key space $\mathcal{K}$ to the domain, yielding a function $KH : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$.

For (unkeyed) hash functions, our security analyses use the standard-model collision resistance (CR), one-wayness (OW), and (generalized) second-preimage resistance (GSPR) properties. When deriving concrete security estimates, however, we model these hash functions as (quantum-accessible) random oracles to invoke the corresponding bounds. For completeness, we summarize these properties and bounds in Appendix A.1.

For KHFs, our analyses consider the *split-key pseudo-random function family* (skPRF) property from [GHP18], a variant of the *pseudo-random function family* (PRF) property where the key space is essentially split into multiple components (or interpreted as such); see Definition 1.

Definition 1 (KHF skPRF). *Given a (split-key) KHF $KH : \mathcal{K}_1 \times \dots \times \mathcal{K}_n \times \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$ playing in the skPRF game (as defined in Fig. 1), the advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{KH}^{\text{skPRF}}(\mathcal{A}) := \max_i (|\Pr[\text{skPRF}_{KH}(\mathcal{A}, 0, i) = 1] - \Pr[\text{skPRF}_{KH}(\mathcal{A}, 1, i) = 1]|) .$$

Game $\text{skPRF}_{KH}(\mathcal{A}, b, i)$:

```

01  $X \leftarrow \emptyset$ 
02  $k_i \leftarrow \mathcal{K}_i$ 
03  $b' \leftarrow \mathcal{A}^{\text{O}_{KH}}()$ 
04 return  $b'$ 

 $\text{O}_{KH}(k_1, \dots, k_{i-1}, k_{i+1}, \dots, k_n, x)$ :
05 if  $x \in X$ 
06   return  $\perp$ 
07  $X \leftarrow X \cup \{x\}$ 
08  $y_0 \leftarrow KH(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_n, x)$ 
09  $y_1 \leftarrow \mathcal{Y}$ 
10 return  $y_b$ 
```

Fig. 1: skPRF game for split-key KHF KH.

Remark 2. [GHP18, Section 4] gives several constructions of skPRFs. In particular, the construction via concatenation ($\|$) and a keyed hash function $KH' : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ —that is, $KH(k_1, \dots, k_n, x) := KH'(k_1 \| \dots \| k_n, x)$ —is extremely common in practice. Modeling KH' as a random oracle, [GHP18, Lemma 6 and Example 3] shows that, for this construction,

$$\text{Adv}_{KH}^{\text{skPRF}}(\mathcal{A}) \leq \frac{q_{KH'}}{|\mathcal{K}|} ,$$

for any adversary $\mathcal{A}$ against skPRF of KH, issuing at most $q_{KH'}$ queries to KH' and q_e queries to O_{KH} .

2.2 Public-Key Encryption

Central to the constructions considered in this work are PKE schemes; we provide the formal definition below.

Definition 3 (Public-Key Encryption Scheme). *A public-key encryption scheme is a triple of algorithms $(\text{KeyGen}, \text{Enc}, \text{Dec})$ defined with respect to public-key space $\mathcal{PK}$, secret-key space $\mathcal{SK}$, message (plaintext) space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$. More precisely, the triple of algorithms is defined as follows.*

- $\text{KeyGen}() : (\text{pk}, \text{sk}) \in \mathcal{PK} \times \mathcal{SK}$. Probabilistic algorithm that takes no inputs and outputs a public key and a secret key (“key pair”).
- $\text{Enc}(\text{pk} \in \mathcal{PK}, m \in \mathcal{M}) : c \in \mathcal{C}$. Probabilistic algorithm that, given a public key pk and message m , produces an encryption (ciphertext) c of m under pk . We may explicitly specify the randomness used in encryption by writing $\text{Enc}(\text{pk}, m; r)$, where $r \in \mathcal{R}$.
- $\text{Dec}(c \in \mathcal{C}, \text{sk} \in \mathcal{SK}) : m \in \mathcal{M} \cup \{\perp\}$. Deterministic algorithm that, given a secret key sk and ciphertext c , yields either a message $m \in \mathcal{M}$ or an explicit failure symbol $\perp \notin \mathcal{M}$, the latter indicating that c is not a valid ciphertext (under sk).

A PKE property used in this work is *rigidity*, introduced by Bernstein and Persichetti [BP18]. This property applies to deterministic PKE schemes, i.e., schemes with deterministic encryption algorithm Enc . Intuitively, such a scheme is *rigid* if Enc is a left-inverse of Dec : decrypting and then re-encrypting always recovers the original ciphertext. Definition 4 formalizes this notion.

Definition 4 (PKE Rigidity [BP18]). *A deterministic PKE scheme $\text{dPKE} := (\text{KeyGen}, \text{Enc}, \text{Dec})$ is rigid if (and only if) for any key pair (pk, sk) generated by KeyGen and any ciphertext $c \in \mathcal{C}$,*

$$\text{Dec}(\text{sk}, c) = \perp \quad \text{or} \quad \text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) = c.$$

2.3 Key Encapsulation Mechanisms

Closely related to the notion of a PKE scheme is that of a KEM. Indeed, many KEM constructions build atop PKE schemes, including several considered in this work. The formal definition is provided below, where we reuse the notation and naming conventions established for PKE schemes when appropriate, for consistency.

Definition 5 (Key Encapsulation Mechanism). *A key encapsulation mechanism is a triple of algorithms $(\text{KeyGen}, \text{Encap}, \text{Decap})$ defined with respect to public-key space $\mathcal{PK}$, secret-key space $\mathcal{SK}$, (shared) key space $\mathcal{K}$, encapsulation/ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$. More precisely, the triple of algorithms is defined as follows.*

- $\text{KeyGen}() : (\text{pk}, \text{sk}) \in \mathcal{PK} \times \mathcal{SK}$. Probabilistic algorithm that takes no inputs and outputs a public key and a secret key (“key pair”).
- $\text{Encap}(\text{pk} \in \mathcal{PK}) : (k, c) \in \mathcal{K} \times \mathcal{C}$. Probabilistic algorithm that, given a public key pk , produces a key k and a corresponding encapsulation (or ciphertext) c under pk . We may explicitly specify the randomness used in encapsulation by writing $\text{Encap}(\text{pk}; r)$, where $r \in \mathcal{R}$.
- $\text{Decap}(c \in \mathcal{C}, \text{sk} \in \mathcal{SK}) : k \in \mathcal{K} \cup \{\perp\}$. Deterministic algorithm that, given a secret key sk and ciphertext c , yields either a $k \in \mathcal{K}$ or an explicit failure symbol $\perp \notin \mathcal{K}$, the latter indicating that c is not a valid encapsulation (under sk).

KEMs, like PKE schemes, typically satisfy a perfect correctness condition: Assuming proper inputs, decapsulation invariably recovers the original (shared) key. However, many post-quantum KEMs (including many FO-based ones) do not satisfy this property. Instead, they permit a small probability of a decapsulation failure, even for honestly generated encapsulations. As such, we consider the looser δ -correctness notion given below, taken from [HHK17].

Definition 6 (KEM δ -correctness [HHK17]). A KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ is δ -correct if

$$\Pr \left[\text{Decap}(\text{sk}, c) \neq k \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(); \\ (k, c) \leftarrow \text{Encap}(\text{pk}) \end{array} \right] \leq \delta .$$

Beyond correctness, we adopt the standard security notion for KEMs, *indistinguishability under chosen-ciphertext attacks* (IND-CCA), jointly formalized by the game and advantage definition provided in Fig. 2 and Definition 7, respectively.

Definition 7 (KEM IND-CCA). Given a KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ and an adversary $\mathcal{A}$ playing in the IND-CCA game (as defined in Fig. 2), the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) := |\Pr[\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 0) = 1] - \Pr[\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 1) = 1]| .$$

Game $\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, b)$:	$\text{DO}_{\text{Decap}}(c)$:
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	07 if $c = c^*$
02 $(k^*, c^*) \leftarrow \text{Encap}(\text{pk})$	08 return $\perp$
03 if b	09 $k \leftarrow \text{Decap}(\text{sk}, c)$
04 $k^* \leftarrow_{\$} \mathcal{K}$	10 return k
05 $b' \leftarrow \mathcal{A}^{\text{DO}_{\text{Decap}}}(\text{pk}, c^*, k^*)$	
06 return b'	

Fig. 2: IND-CCA game for KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$.

Further, we use a less conventional security property called *ciphertext second-preimage resistance* (C2PRI), introduced in [BCD⁺24] for the IND-CCA analysis of the hybrid KEM X-Wing. Intuitively, C2PRI denotes the difficulty of finding an encapsulation that decapsulates to the same key as a *different* honestly generated encapsulation, even when the secret decapsulation key is *fully* exposed—effectively nullifying any conventional KEM security guarantees such as IND-CCA. The rationale for this powerful adversarial model is the presumed scenario in which the notion is used: a completely insecure KEM (in the sense of IND-CCA). Indeed, providing the decapsulation key to the adversary is a way to model this scenario. Formally, C2PRI is defined as in Definition 8.

Definition 8 (KEM C2PRI [BCD⁺24]). Given a KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ and an adversary $\mathcal{A}$ playing in the C2PRI game (as defined in Fig. 3), the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) := \Pr[\text{C2PRI}_{\text{KEM}}(\mathcal{A}) = 1] .$$

Lastly, we consider a class of *binding properties*, put forward by Cremers et al. in [CDM24]. These properties primarily serve to prevent attack scenarios arising in protocols that traditional properties like IND-CCA do not cover. Loosely speaking, a binding property captures to what extent certain KEM artifacts determine others. Here, the focus is on the artifacts known to both parties in a KEM interaction: the public key, the encapsulation, and the shared key. For example, if the shared key *binds* the encapsulation, then it is hard to find *distinct* encapsulations that successfully decapsulate to the *same* shared key. In other words, fixing the shared key “sufficiently restricts” the set of

Game C2PRI_{KEM}($\mathcal{A}$): 01 $(pk, sk) \leftarrow \text{KeyGen}()$ 02 $(k, c) \leftarrow \text{Encap}(pk)$ 03 $c' \leftarrow \mathcal{A}(sk, pk, k, c)$ 04 return $\text{Decap}(sk, c') = k \wedge c' \neq c$

Fig. 3: C2PRI game for $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$.

encapsulations that lead to successful decapsulation. For ease of reference, when discussing whether P binds Q , we refer to elements of P and Q as *binding sources* and *binding targets*, respectively.

The possible choices of binding sources and targets give rise to six (non-trivial) binding properties. Cremers et al. also distinguish three adversarial models: honest (HON), leaking (LEAK), and malicious (MAL). In HON, the adversary has access to a decapsulation oracle, as in IND-CCA. In LEAK, it instead learns the secret decapsulation keys, making such an oracle unnecessary. In MAL, it may choose the secret decapsulation keys itself. In total, this yields 18 binding properties, which we denote by $X\text{-BIND-}P\text{-}Q$ where $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$ and $P, Q \subset \{\text{PK}, \text{C}, \text{K}\}$ (such that $P \cap Q = \emptyset$). Here, PK, C, and K respectively signify public key, encapsulation, and shared key. Further, for readability, we often omit the explicit set notation for P and Q in property names.

The formal definitions of these binding properties are quite alike. As such, instead of providing dedicated definitions for each of them, we provide two slightly more generic definitions, closely following the original work [CDM24]. For notational convenience, we also let $\text{PK} := \{pk_0, pk_1\}$, $\text{C} := \{c_0, c_1\}$, and $\text{K} := \{k_0, k_1\}$. Moreover, although we establish results for all 18 properties for both combinators, the main exposition focuses on the LEAK model. Accordingly, we give its formal definition (which also covers HON) here and defer the definition of MAL to Appendix A.2.

Definition 9 (KEM Binding (HON and LEAK) [CDM24]). Let $X \in \{\text{HON}, \text{LEAK}\}$ and $P, Q \subset \{\text{PK}, \text{C}, \text{K}\}$ such that $P \cap Q = \emptyset$, where $\text{PK} := \{pk_0, pk_1\}$, $\text{C} := \{c_0, c_1\}$, and $\text{K} := \{k_0, k_1\}$. Further, define “binding break” predicate

$$\begin{aligned} \text{BB}_{P,Q} : \mathcal{PK} \times \mathcal{PK} \times \mathcal{C} \times \mathcal{C} \times \mathcal{K} \times \mathcal{K} &\rightarrow \{\text{true}, \text{false}\} \\ \text{BB}_{P,Q}(pk_0, pk_1, c_0, c_1, k_0, k_1) &:= \\ k_0 \neq \perp \wedge k_1 \neq \perp \wedge \forall_{p \in P} : p_0 = p_1 \wedge \exists_{q \in Q} : q_0 \neq q_1 &, \end{aligned}$$

where p_b (resp. q_b) refers to the respective element in the set p (resp. q); e.g., if $p = \text{PK}$, p_0 denotes pk_0 . Then, given a KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ and an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ playing in the $X\text{-BIND-}P\text{-}Q$ game (as defined in Fig. 4), the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{KEM}}^{X\text{-BIND-}P\text{-}Q}(\mathcal{A}) := \Pr[X\text{-BIND-}P\text{-}Q_{\text{KEM}}(\mathcal{A}) = 1] .$$

2.4 Fujisaki-Okamoto Transformation

The FO transform—or, rather, the modular revision accompanied by a quantum-security analysis [HHK17]—constructs a KEM from a PKE scheme in a modular fashion. It does so by first applying the T transform to the given PKE scheme, resulting in a derandomized version, and subsequently subjecting this derandomized scheme to the U transform to produce the final KEM. In fact, the U transform has four main variants, leading to a total of four FO variants.

Loosely speaking, the T transform “generates randomness” from the given message via a hash function. Further, it adds a re-encryption step to decryption, failing if the message extracted from

```

Game X-BIND-P-QKEM( $\mathcal{A}$ ):
01  $(pk_0, sk_0) \leftarrow \text{KeyGen}()$ 
02  $(pk_1, sk_1) \leftarrow \text{KeyGen}()$ 

03 if  $PK \in P$ :  $b \leftarrow 0$ 
04 else if  $PK \in Q$ :  $b \leftarrow 1$ 
05 else:  $b \leftarrow \mathcal{A}_1()$ 

06  $(pk_1, sk_1) \leftarrow (pk_b, sk_b)$ 

07 if  $X = \text{HON}$ 
08    $(c_0, c_1) \leftarrow \mathcal{A}_2^{\text{DO}_{\text{Decap}}}(pk_0, pk_1)$ 
09 else if  $X = \text{LEAK}$ 
10    $(c_0, c_1) \leftarrow \mathcal{A}_2(pk_0, sk_0, pk_1, sk_1)$ 

11  $k_0 \leftarrow \text{Decap}(sk_0, c_0)$ 
12  $k_1 \leftarrow \text{Decap}(sk_1, c_1)$ 
13 return  $\text{BB}_{P,Q}(pk_0, pk_1, c_0, c_1, k_0, k_1)$ 

DODecap( $b', c$ ):
14  $k \leftarrow \text{Decap}(sk_{b'}, c)$ 
15 return  $k$ 

```

Fig. 4: X-BIND-P-Q game for $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$, $X \in \{\text{HON}, \text{LEAK}\}$, and $P, Q \subset \{PK, C, K\}$ where $P \cap Q = \emptyset$.

the ciphertext does not result in the same ciphertext when encrypted anew. The $\mathbf{U}$ transform produces a shared key and corresponding encapsulation by encrypting a randomly sampled message, subsequently passing it through a hash function. (Then, the hash digest and ciphertext serve as the key and encapsulation, respectively.) The variants of this transform differ along two dimensions, each allowing for two possible modulus operandi. The first dimension concerns the derivation of the key, which may be done by passing either (1) only the message or (2) both the message and its encryption through the hash function. The second dimension concerns the response in case of a failure, which may be either (1) $\perp$ or (2) a pseudo-random value. Typically, for this second dimension, these are referred to as *explicit* and *implicit rejection*, respectively. The formal definitions of the transforms are provided below.¹⁰

Definition 10 (T Transform). *Given a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and a hash function $G : \mathcal{M} \rightarrow \mathcal{R}$, the $\mathbf{T}$ transform constructs the PKE scheme*

$$\mathbf{T}[\text{PKE}, G] := (\text{KeyGen}', \text{Enc}', \text{Dec}') ,$$

as defined in Fig. 5.

Remark 11. The $\mathbf{T}$ transform transforms any probabilistic PKE into a rigid deterministic PKE because re-encryption explicitly verifies the rigidity condition.

Definition 12 ($\mathbf{U}^\perp$ and $\mathbf{U}^\mathcal{K}$ Transforms). *Given a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and hash functions $H : \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{K}$ and $F : \mathcal{K}_F \times \mathcal{C} \rightarrow \mathcal{K}$, the $\mathbf{U}^\perp$ transform and $\mathbf{U}^\mathcal{K}$ transforms respectively*

¹⁰ For the implicitly rejecting $\mathbf{U}$ transforms, we explicitly include a dedicated function for generating rejection keys, as done in several post-quantum KEMs (though not technically in [HHK17]).

KeyGen' ($\cdot$):	Enc' (pk, m):	Dec' ($\text{sk}' := (\text{pk}, \text{sk}), c$):
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	04 $c \leftarrow \text{Enc}(\text{pk}, m; \text{G}(m))$	06 $m \leftarrow \text{Dec}(\text{sk}, c)$
02 $\text{sk}' \leftarrow (\text{pk}, \text{sk})$	05 return c	07 if $m = \perp$ or $c \neq \text{Enc}'(\text{pk}, m)$
03 return (pk, sk')		08 return $\perp$
		09 else
		10 return m

Fig. 5: T-transformed encryption scheme $\text{T}[\text{PKE}, \text{G}]$.

KeyGen' $^\perp$ ($\cdot$)	Encap $_{\overline{m}}$ (pk)
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	09 $m \leftarrow \mathcal{M}$
02 $\text{fk} \leftarrow \mathcal{K}_F$	10 $c \leftarrow \text{Enc}(\text{pk}, m)$
03 $\text{sk}' \leftarrow (\text{sk}, \text{fk})$	11 $k \leftarrow \text{H}(m, c) \boxed{\text{H}(m)}$
04 return (pk, sk')	12 return (k, c)
Decap $_{\overline{m}}^\perp$ (sk, c)	Decap $_{\overline{m}}'$ ($\text{sk}' := (\text{sk}, \text{fk}), c$)
05 $m \leftarrow \text{Dec}(\text{sk}, c)$	13 $m \leftarrow \text{Dec}(\text{sk}, c)$
06 if $m = \perp$	14 if $m = \perp$
07 return $\perp$	15 return $\text{F}(\text{fk}, c)$
08 return $\text{H}(m, c) \boxed{\text{H}(m)}$	16 return $\text{H}(m, c) \boxed{\text{H}(m)}$

Fig. 6: $\text{U}^\perp[\text{PKE}, \text{H}]$, $\text{U}^\perp[\text{PKE}, \text{H}, \text{F}]$, $\text{U}_m^\perp[\text{PKE}, \text{H}]$ and $\text{U}_m^\perp[\text{PKE}, \text{H}, \text{F}]$. Outlined code belongs to the latter two transforms only and, for these transforms, replaces the directly preceding code.

construct the KEMs

$$\begin{aligned}
\text{U}^\perp[\text{PKE}, \text{H}] &:= (\text{KeyGen}, \text{Encap}, \text{Decap}^\perp) , \\
\text{U}^\perp[\text{PKE}, \text{H}, \text{F}] &:= (\text{KeyGen}^\perp, \text{Encap}, \text{Decap}^\perp) , \\
\text{U}_m^\perp[\text{PKE}, \text{H}] &:= (\text{KeyGen}, \text{Encap}_m, \text{Decap}_m^\perp) , \\
\text{U}_m^\perp[\text{PKE}, \text{H}, \text{F}] &:= (\text{KeyGen}^\perp, \text{Encap}_m, \text{Decap}_m^\perp) ,
\end{aligned}$$

as defined in Fig. 6.

2.5 KEM Combiners, QSF, and X-Wing

Broadly speaking, KEM combiners are generic frameworks for constructing a single KEM from multiple constituent KEMs; the resulting KEM is supposed to be secure—typically in the sense of IND-CCA—if *at least one* of the input KEMs is secure. In many cases, such combiners directly preserve the security of their inputs—that is, the resulting KEM is at least as secure as any of its constituents. This property makes them especially attractive for building PQ/T hybrid KEMs: by combining a pre- and post-quantum KEM, classical security can be guaranteed even if the post-quantum component KEM turns out insecure. Moreover, it enables certification of post-quantum systems under guidelines that still require traditional algorithms.

In [GHP18], Giacon et al. conducted a detailed study of such KEM combiners, focusing on IND-CCA security. Later, with the goal of gaining both efficiency and (engineering) simplicity in the hybrid setting, Barbosa et al. proposed the QSF framework and, specifically, its instantiation X-Wing [BCD⁺24]. Unlike customary combiners, QSF takes a KEM and a *nominal group* as input. If

the component KEM is (post-quantum) IND-CCA secure, then the resulting KEM is as well—even if the nominal group is completely insecure. Moreover, if the constituent KEM is only C2PRI secure but the nominal group satisfies the Strong Diffie-Hellman (SDH) assumption, then the resulting KEM is still (pre-quantum) IND-CCA secure. The X-Wing scheme instantiates QSF with ML-KEM-768 as the KEM and X25519 as the nominal group.

On a high level, QSF operates by executing the provided KEM and nominal group—say KEM and $\mathcal{N}$ —in parallel. For key generation, it generates key pairs using both KEM and $\mathcal{N}$, setting its own key pair to be the component-wise pair:

$$(\text{pk}_0, \text{sk}_0) \leftarrow \text{KEM.KeyGen}; (\text{pk}_1, \text{sk}_1) \leftarrow \mathcal{N}.\text{KeyGen}; (\text{pk}, \text{sk}) \leftarrow ((\text{pk}_0, \text{pk}_1), (\text{sk}_0, \text{sk}_1)) .$$

For encapsulation, it encapsulates using KEM, exponentiates the group generator and (the relevant part of) the public key by a freshly generated ephemeral secret using $\mathcal{N}$ —the resulting values respectively serve as “group-based encapsulation and shared secret”—deriving its own shared secret as the hash of the obtained shared secrets (along with the group-based public key, encapsulation, and an algorithm-specific label), and setting its own encapsulation to be the concatenation of the obtained encapsulations.¹¹

$$(k_0, c_0) \leftarrow \text{KEM.Encap}(\text{pk}_0); (k_1, c_1) \leftarrow (g^{\text{sk}_e}, \text{pk}_1^{\text{sk}_e}); (k, c) \leftarrow (\text{H}(k_0, k_1, c_1, \text{pk}_1, \text{algotlabel}), (c_0, c_1)) .$$

For decapsulation, it decapsulates using KEM, exponentiates (the relevant part of) the encapsulation using $\mathcal{N}$, and derives the shared secret as in encapsulation:

$$k_0 \leftarrow \text{KEM.Decap}(\text{sk}_0, c_0); k_1 \leftarrow c_1^{\text{sk}_1}; k \leftarrow \text{H}(k_0, k_1, c_1, \text{pk}_1, \text{algotlabel}) .$$

For completeness, we provide the formal definition of nominal groups and the full specification of QSF in Section A.3.

Remark 13. Prime-order groups, such as prime-order elliptic curves, are instances of nominal groups [ABH⁺21], as shown by Alwen, Blanchet, Hauck, Kiltz, Lipp, and Riepel. As such, under the SDH assumption, the curves brainpoolP256, brainpoolP384, and brainpoolP512 [ML10], as well as NIST’s P-256, P-384, and P-521 [oST23] can be used to instantiate QSF’s nominal group (as ECDH).

3 QSI: A KEM-Only Framework

As briefly mentioned before, the QSF framework—and particularly the instantiation as X-Wing (using ML-KEM-768 and X25519)—introduced in [BCD⁺24] achieves pre- and/or post-quantum security by combining a KEM and a nominal group: if the KEM is (post-quantum) IND-CCA secure, QSF is (post-quantum) IND-CCA secure regardless of the security of the nominal group; if the nominal group satisfies the SDH assumption and the KEM provides no security beyond C2PRI, then QSF is (pre-quantum) IND-CCA secure. In the ensuing, we look into combining *two arbitrary KEMs* instead, following ideas used in QSF to optimize combiner inputs. That is, we introduce a KEM-only variant of QSF, which we call Quantum Superiority Interceptor (QSI), and analyze its IND-CCA *and* binding security.

Conceptually, our framework is almost identical to QSF, merely replacing all nominal group operations by the appropriate KEM operations and deriving the final shared key in a (slightly) different manner. Specifically, while QSF includes (1) the shared key from its KEM and (2) the shared key, encapsulation/ciphertext, and public encapsulation key from the nominal group (and a fixed label), our framework only includes the shared keys from both KEMs (and a fixed label),

¹¹ Here, g denotes the group generator and sk_e denotes the freshly generated ephemeral secret.

KeyGen()	Encap(pk := (pk ₀ , pk ₁))	Decap(sk := (sk ₀ , sk ₁), c := (c ₀ , c ₁))
01 (pk ₀ , sk ₀) ← KEM ₀ .KeyGen()	06 (k ₀ , c ₀) ← KEM ₀ .Encap(pk ₀)	11 k ₀ ← KEM ₀ .Decap(sk ₀ , c ₀)
02 (pk ₁ , sk ₁) ← KEM ₁ .KeyGen()	07 (k ₁ , c ₁) ← KEM ₁ .Encap(pk ₁)	12 k ₁ ← KEM ₁ .Decap(sk ₁ , c ₁)
03 pk ← (pk ₀ , pk ₁)	08 k ← H(k ₀ , k ₁ , algolabel)	13 if k ₀ = ⊥ or k ₁ = ⊥
04 sk ← (sk ₀ , sk ₁)	09 c ← (c ₀ , c ₁)	14 return ⊥
05 return (pk, sk)	10 return (k, c)	15 return H(k ₀ , k ₁ , algolabel)

Fig. 7: QSI[KEM₀, KEM₁, H].

thereby reducing the hashing workload (see Appendix B for a discussion of the resulting performance impact). Usually, the public encapsulation key is included to ensure multi-user security. However, we argue this is unnecessary for QSI as it plausibly inherits such security from its constituent KEMs. The full specification of the framework is given in Fig. 7.

SECURITY RESULTS. We analyze the security of QSI, covering both IND-CCA (Section 3.1) and several binding properties (Section 3.2). For comparison, we include an analysis of the binding properties for QSF, which, to the best of our knowledge, has not been done before.

We show that QSI provides (pre- and/or post-quantum) IND-CCA security as long as one of the constituent KEMs is (pre- and/or post-quantum) IND-CCA and the other is (pre- and/or post-quantum) C2PRI secure. In other words, we demonstrate the IND-CCA security of QSI in multiple scenarios, including the possibility of cryptographically relevant quantum computers becoming a reality—nullifying the IND-CCA security of pre-quantum KEMs—and the possibility of post-quantum cryptography being classically broken, such as by an attack, vulnerability, or implementation error—nullifying the IND-CCA security of post-quantum KEMs. More precisely, in the former scenario, where the pre-quantum constituent KEM provides (classical) IND-CCA security but the post-quantum constituent KEM may not offer any IND-CCA security at all, our result implies that QSI is (pre-quantum) IND-CCA secure when

1. the key-derivation function H is a split-key PRF;
2. the pre-quantum KEM provides IND-CCA security; and
3. the post-quantum KEM provides no security beyond C2PRI.¹²

In the latter scenario, where the post-quantum constituent KEM provides (post-quantum) IND-CCA security but the pre-quantum KEM may not offer any IND-CCA security at all, our result implies that QSI is (post-quantum) IND-CCA secure when

1. the key-derivation function H is a split-key PRF;
2. the pre-quantum KEM provides no security beyond C2PRI; and
3. the post-quantum KEM provides IND-CCA security.¹³

Notably, this result does *not* rely on the (Q)ROM: it is sufficient to assume the standard model skPRF property for the employed hash function H.

Further, we establish that *all considered variants* of the FO transform provide C2PRI security. Together with the IND-CCA guarantees of these transforms, this makes FO-based KEMs quite suitable as constituent KEMs for QSI.

We also briefly consider a variant of QSI that uses the encapsulations of its (second) constituent KEM as input to the final key derivation, akin to QSF. We show that the variant is IND-CCA-secure

¹² This case would consider classical attackers for all properties.

¹³ This case would consider quantum attackers for all properties.

(in the ROM) already if the second KEM satisfies OW-PCVA, a notion that is weaker than IND-CCA. In essence, OW-PCVA is an abstraction of SDH for KEMs. As such, a QROM analysis would not be particularly useful, and we only consider the (*classical*) ROM. However, because OW-PCVA-based results are not the main focus of this work, we defer this analysis to the appendix (Section C.1).

For both QSI and QSF, we establish results for all six considered (see Section 2.3) binding properties across the three models. Most of these are preservation results, capturing the sense in which the combiners preserve the binding properties of their constituent KEM(s). Here, for QSI, we consider different “kinds” of preservation, which differ in the requirements on the constituent KEMs; for QSF, there is no such distinction. Additionally, for QSF, we establish some standalone results, demonstrating that the combiner satisfies certain binding properties without relying on any such property of its constituent KEM. Lastly, we show that QSF does *not* (necessarily) preserve HON- and LEAK-BIND-CT-PK, and does *not* satisfy MAL-BIND-CT-K or MAL-BIND-CT-PK.¹⁴ Broadly speaking, the differences in the results for QSF and QSI primarily arise due to the fact that (1) QSF uses all artifacts from its nominal group as inputs to the final key derivation, and (2) QSI consists of two KEMs, particularly meaning we can rely on the potential binding properties for either (or both).

HASHING THE PUBLIC KEYS/ENCAPSULATIONS FOR MULTI-USER/CHALLENGE SECURITY? In practice, regular IND-CCA security might be deemed insufficient: KEMs are employed in applications with many users, each of which may use their KEM multiple times over, and so attackers could observe/store interactions over an extended period of time before exploiting the collected public keys and encapsulations. As such, it is desirable to guarantee IND-CCA-like security even in this scenario, which leads to a multi-user/challenge version of IND-CCA. For FO-based KEMs, a known approach for achieving multi-user security is to tie each challenge to its user by hashing the user’s public key into the challenge key [DHK⁺21]. Naturally, one might think that QSI can achieve multi-user security through the same adaptation, and possibly multi-challenge security by additionally hashing the encapsulations. This would mean changing the key derivation to

$$H(k_0, k_1, c_0, c_1, \text{pk}_0, \text{pk}_1, \text{algolabel}) ,$$

increasing the size of the hash input significantly. Fortunately, it seems that the additional hash inputs may be unnecessary: QSI inherently preserves the multi-user/challenge IND-CCA of its constituents. While multi-user/challenge security is not a central topic of this work, we provide a proof sketch for this claim in Section C.2.

HASHING THE PUBLIC KEYS/ENCAPSULATIONS FOR (BETTER) BINDING? As noted above, the binding results established for QSF do not rely on any binding-like properties of its nominal group. This primarily stems from QSF’s use of the nominal group’s public key and encapsulation as inputs to the key derivation, which hints at an underlying pattern: if the key-derivation function is collision resistant, passing the public key and/or encapsulation of a constituent KEM into the (final) key derivation removes the dependency on that KEM’s binding properties, at least for those with the shared key as binding source. In essence, this is a variant of the post-processing remark made by Cremers et al. [CDM24], but for combiners like the ones considered here.

The tradeoff, however, is that it requires passing additional data into the final key derivation—especially for some of the post-quantum KEMs, this can be costly (cf. Appendix B). As such, depending on the (relative) practicality of (1) instantiating the combiner with KEMs that satisfy the desired binding properties and (2) passing in the public key and/or encapsulation to the final key derivation, one approach might be preferred over the other, but this is context-dependent. Furthermore, adding more inputs to the final key derivation can also *preclude* certain binding properties

¹⁴ Contrarily, for QSI, we do obtain positive results for these properties.

or their preservation. For instance, if the public key of a constituent KEM is included, the resulting combiner may no longer preserve the binding property with the encapsulation as source and the shared key as target, as exemplified by QSF. (For more details, refer to the LEAK-BIND-CT-K analysis of QSF.) So, in contexts where such binding is desired, this may be another consideration.

3.1 Security: IND-CCA

We start by examining the most conventional property, IND-CCA. Loosely speaking, our analysis demonstrates that QSI is IND-CCA secure if one of its constituent KEMs is IND-CCA secure, the other constituent KEM is C2PRI secure, and the employed hash function is an skPRF. This result is symmetric with regard to the constituent KEMs: it is irrelevant which KEM satisfies IND-CCA, as long as the other satisfies C2PRI. We capture both cases with the single theorem below.

Theorem 14 (QSI IND-CCA). *Let $b \in \{0, 1\}$, and let KEM_0 and KEM_1 be KEMs—with respective key spaces $\mathcal{K}_0$ and $\mathcal{K}_1$ —of which KEM_{1-b} is δ -correct. Further, let $\text{algotlabel} \in \mathcal{L}$ be an algorithm-specific label, $H : \mathcal{K}_0 \times \mathcal{K}_1 \times \mathcal{L} \rightarrow \mathcal{K}$ a (split-key) KHF, and $\text{KEM} := \text{QSI}[\text{KEM}_0, \text{KEM}_1, H]$. Then, for any adversary $\mathcal{A}$ against IND-CCA of KEM (as defined in Fig. 2), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{KEM}_b}^{\text{C2PRI}}(\mathcal{B}) + 2 \cdot \text{Adv}_{\text{KEM}_{1-b}}^{\text{IND-CCA}}(\mathcal{C}) + 2 \cdot \text{Adv}_H^{\text{skPRF}}(\mathcal{D}) + 2\delta .$$

Here, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$, $\mathcal{C} := \mathcal{C}(\mathcal{A})$, and $\mathcal{D} := \mathcal{D}(\mathcal{A})$ are as constructed in the proof of this theorem. $\mathcal{C}$ issues at most q_{DO} queries to its decapsulation oracle, and $\mathcal{D}$ issues at most q_{DO} queries to its skPRF oracle.

The proof loosely follows previous KEM combiner proofs, specifically the ones for [GHP18, Thm. 1] and [BCD⁺24, Thm. 2], while integrating the C2PRI-related reasoning used in [BCD⁺24, Thm. 1] for QSF. Moreover, the proofs for the two cases—only differing in which constituent is considered for C2PRI and which for IND-CCA—are completely symmetrical. So, without loss of generality, we cover the case in which KEM_0 is considered for C2PRI and KEM_1 for IND-CCA.

On a high level, the proof proceeds as follows. First, to bound the IND-CCA advantage, we start from the game where the KEM challenge key is honest, meaning $k^* = H(k_0^*, k_1^*, \text{algotlabel})$. Subsequently, we change this game until we arrive at the game in which the KEM challenge key k^* is random. To this end, we rely on the skPRF property of H to argue that $k^* = H(k_0^*, k_1^*, \text{algotlabel})$ is indistinguishable from random. Before we can apply such a reasoning step, however, at least one (split-)key should be uniformly random. For this purpose, we randomize k_1^* , based on the IND-CCA property of KEM_1 . (Here, our approach works up to correctness errors, each of which occurs with probability δ .) Further, to use the skPRF property, we additionally need to ensure that the value $H(k_0^*, k_1^*, \text{algotlabel})$ is never given to $\mathcal{A}$ (as the result of a decapsulation query)—otherwise, the replacement would be noticeable. This could happen if there were other KEM_0 encapsulations that decapsulate to k_0^* —this is where C2PRI comes into play. Lastly, the factors (of 2) arise because we must undo the changes we made to replace the honestly computed k^* with a random one—with these changes still present, we do not yet end up with the proper IND-CCA game for the random key case. (Some decapsulation queries are not yet answered correctly.) Undoing the changes finishes our reasoning. The following proof formalizes this high-level overview.

Proof. We prove the statement via a sequence of games, starting with the original $\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 0)$ (i.e., the IND-CCA game for KEM with a honestly computed challenge key) as G_0 .

$$\Pr[\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 0) = 1] = \Pr[G_0(\mathcal{A}) = 1] .$$

Table 1: Outline of games used in the proof of Theorem 14.

Game	k^*	k_0^*	k_1^*	$\text{Decap}_0(\text{sk}_0, -)$	$\text{Decap}(\text{sk}, - c_1^*)$	Δ
G_0	real	real	real	usual	usual	$\text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}()$
G_1	real	real	real	C2PRI abort	usual	$\text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}()$
G_2	real	real	random	C2PRI abort	usual	$\text{Adv}_{\text{H}}^{\text{skPRF}}()$
G_3	random	real	random	C2PRI abort	random	$\text{Adv}_{\text{H}}^{\text{skPRF}}()$
G_4	random	real	random	C2PRI abort	usual	$\text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}()$
G_5	random	real	real	C2PRI abort	usual	$\text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}()$
G_6	random	real	real	usual	usual	

Game G_1 . We first change the game such that it aborts whenever the challenge key, $k^* = \text{H}(k_0^*, k_1^*, \text{algotlabel})$, could be given to $\mathcal{A}$ through a permitted decapsulation query; concretely, G_1 aborts if $\mathcal{A}$ ever queries the decapsulation oracle on an encapsulation (c_0, c_1^*) such that $c_0 \neq c_0^*$, yet $\text{Decap}(\text{sk}_0, c_0) = \text{Decap}(\text{sk}_0, c_0^*) = k_0^*$. Thus, the game only differs if $\mathcal{A}$ performs a successful C2PRI attack on KEM_0 . We turn this into a successful C2PRI reduction $\mathcal{B}_1$. Since C2PRI attackers on KEM_0 get the secret key used by KEM_0 and can generate the key pair for KEM_1 on their own, $\mathcal{B}_1$ can perfectly simulate the decapsulation oracle and immediately recognize and output any C2PRI break (c_0) that might appear in $\mathcal{A}$'s decapsulation queries. Hence, we can bound the difference in success probability between G_0 and G_1 as

$$|\Pr[G_0(\mathcal{A}) = 1] - \Pr[G_1(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}(\mathcal{B}_1) .$$

Games $G_0 - G_2$:	$\text{DO}_{\text{Decap}}(c := (c_0, c_1))$:
01 $(\text{pk}_0, \text{sk}_0) \leftarrow \text{KEM}_0.\text{KeyGen}()$	09 if $c = c^*$
02 $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KEM}_1.\text{KeyGen}()$	10 return $\perp$
03 $(k_0^*, c_0^*) \leftarrow \text{KEM}_0.\text{Encap}(\text{pk}_0)$	11 $k_0 \leftarrow \text{KEM}_0.\text{Decap}(\text{sk}_0, c_0)$
04 $(k_1^*, c_1^*) \leftarrow \text{KEM}_1.\text{Encap}(\text{pk}_1)$	12 $k_1 \leftarrow \text{KEM}_1.\text{Decap}(\text{sk}_1, c_1)$
05 $k_1^* \leftarrow \mathcal{K}_1$ // G_2	13 if $k_0 = \perp$ or $k_1 = \perp$
06 $k^* \leftarrow \text{H}(k_0^*, k_1^*, \text{algotlabel})$	14 return $\perp$
07 $b' \leftarrow \mathcal{A}^{\text{DO}_{\text{Decap}}}((\text{pk}_0, \text{pk}_1), (c_0^*, c_1^*), k^*)$	15 if $c_0 \neq c_0^*$ and $k_0 = k_0^*$ and $c_1 = c_1^*$ // $G_1 - G_2$
08 return b'	16 abort // $G_1 - G_2$
	17 if $c_1 = c_1^*$ // G_2
	18 $k_1 \leftarrow k_1^*$ // G_2
	19 $k \leftarrow \text{H}(k_0, k_1, \text{algotlabel})$
	20 return k

Fig. 8: Games $G_0 - G_2$ for the proof of Theorem 14.

Game G_2 . In G_2 , we replace the shared key of KEM_1 , k_1^* , with a uniformly random value. To bound the change in success probability, we construct a first IND-CCA adversary $\mathcal{C}_1$ against KEM_1 that simulates the entire game and outputs $\mathcal{A}$'s guess. $\mathcal{C}_1$ generates its own KEM_0 keypair and uses its KEM_1 decapsulation oracle to simulate all decapsulation queries, except when the KEM_1 encapsulation equals the challenge c_1^* , which yields nothing useful ($\perp$) when queried. In that case, it uses the received challenge key k_1^* to simulate the response. This simulation is perfect unless c_1^* fails to decapsulate to k_1^* , which happens with probability at most δ , the correctness error of KEM_1 .

Since $\mathcal{C}_1$ perfectly simulates G_2 if it was given a uniformly random shared key k_1^* , and perfectly simulates G_1 if it was given an honestly encapsulated shared key k_1^* (up to a correctness error), we obtain

$$|\Pr[G_1(\mathcal{A}) = 1] - \Pr[G_2(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}(\mathcal{C}_1) + \delta .$$

Games $G_2 - G_5$:		$\text{DO}_{\text{Decap}}(c := (c_0, c_1))$:	
01 $(\text{pk}_0, \text{sk}_0) \leftarrow \text{KEM}_0.\text{KeyGen}()$		10 if $c = c^*$	
02 $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KEM}_1.\text{KeyGen}()$		11 return $\perp$	
03 $(k_0^*, c_0^*) \leftarrow \text{KEM}_0.\text{Encap}(\text{pk}_0)$		12 $k_0 \leftarrow \text{KEM}_0.\text{Decap}(\text{sk}_0, c_0)$	
04 $(k_1^*, c_1^*) \leftarrow \text{KEM}_1.\text{Encap}(\text{pk}_1)$		13 $k_1 \leftarrow \text{KEM}_1.\text{Decap}(\text{sk}_1, c_1)$	
05 $k_1^* \leftarrow \mathcal{K}_1$	// G_2, G_4	14 if $k_0 = \perp$ or $k_1 = \perp$	
06 $k^* \leftarrow \text{H}(k_0^*, k_1^*, \text{algotlabel})$	// G_2	15 return $\perp$	
07 $k^* \leftarrow \mathcal{K}$	// $G_3 - G_5$	16 if $c_0 \neq c_0^*$ and $k_0 = k_0^*$ and $c_1 = c_1^*$	
08 $b' \leftarrow \mathcal{A}^{\text{DO}_{\text{Decap}}}((\text{pk}_0, \text{pk}_1), (c_0^*, c_1^*), k^*)$		17 abort	
09 return b'		18 if $c_1 = c_1^*$	// G_2, G_4
		19 $k_1 \leftarrow k_1^*$	// G_2, G_4
		20 $k \leftarrow \text{H}(k_0, k_1, \text{algotlabel})$	
		21 if $c_1 = c_1^*$	// G_3
		22 $k \leftarrow \mathcal{K}$	// G_3
		23 return k	

Fig. 9: Games $G_2 - G_5$ for the proof of Theorem 14.

Game G_3 . We now change the game such that all shared keys for KEM are replaced with random values *if* they would have otherwise been computed using the shared key for KEM_1 , k_1^* . This includes the challenge shared key k^* , as well as shared keys computed during decapsulation whenever the KEM_1 part of the queried encapsulation is c_1^* . We can bound the difference between the two success probabilities using a first skPRF adversary $\mathcal{D}_1$ against H that simulates the game, computes the challenge shared key via a call to its skPRF oracle, and returns the game's output as its own bit. To simulate the decapsulation oracle, $\mathcal{D}_1$ generates key pairs for KEM_0 and KEM_1 , and uses them to handle all encapsulations *except* those where the KEM_1 part is c_1^* . For those, $\mathcal{D}_1$ decapsulates the other part, computing $k_0 := \text{Decap}(\text{sk}_0, c_0)$, and queries its skPRF oracle on the input $(k_0, \text{algotlabel})$ to obtain the final shared key.

Since $\mathcal{D}_1$ perfectly simulates game G_2 if its skPRF oracle uses H , and perfectly simulates game G_3 if its skPRF oracle uses random values, it follows that

$$|\Pr[G_2(\mathcal{A}) = 1] - \Pr[G_3(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{H}}^{\text{skPRF}}(\mathcal{D}_1) .$$

Game G_4 . We now keep the challenge secret k^* random, but undo the change of the decapsulation oracle that was made in game G_3 . That is, we change the game back such that the decapsulation oracle no longer uses random values if the KEM_1 part of the queried encapsulation is c_1^* . To bound

the difference between the two success probabilities, we construct a second skPRF adversary $\mathcal{D}_2$ against $\mathbf{H}$ that behaves exactly like $\mathcal{D}_1$, except that it picks the challenge shared key k^* on its own, at random. We obtain

$$|\Pr[G_3(\mathcal{A}) = 1] - \Pr[G_4(\mathcal{A}) = 1]| \leq \text{Adv}_{\mathbf{H}}^{\text{skPRF}}(\mathcal{D}_2) .$$

At this point, the game is almost identical to $\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 1)$, the game we wanted to end up with—except that (1) the KEM_1 shared key, k_1^* , is still replaced with uniformly random values, and (2) the game is still aborted when the honest shared key, $k^* = \mathbf{H}(k_0^*, k_1^*, \text{algotlabel})$, could be given to the attacker through a permitted decapsulation query. As such, we now undo these changes.

Game G_5 . We change the game by reverting k_1^* back to being the honest shared KEM_1 secret, and by dropping the usage of the challenge k_1^* when handling c_1^* (during decapsulation). To bound the difference between the two success probabilities, we construct a second IND-CCA adversary $\mathcal{C}_2$ against KEM_1 that simulates the game and returns the game's output as its own bit. To simulate the decapsulation oracle, $\mathcal{C}_2$ picks its own key pair for KEM_0 and uses its own decapsulation oracle to decapsulate the KEM_1 encapsulations. To handle c_1^* , $\mathcal{C}_2$ will use its own challenge shared key k_1^* .

To relate the reduction's distinguishing advantage to the difference between the two success probabilities, note that $\mathcal{C}_2$ perfectly emulates G_4 if it was given a uniformly random shared key k_1^* ; and $\mathcal{C}_2$ perfectly emulates G_5 if it was given an honestly encapsulated shared key k_1^* , unless the encapsulation c_1^* fails to decapsulate to k_1^* . Again, this happens with probability at most δ . As such, we get

$$|\Pr[G_4(\mathcal{A}) = 1] - \Pr[G_5(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}(\mathcal{C}_2) + \delta .$$

Games $G_5 - G_6$:	DO_{Decap}($c := (c_0, c_1)$):
01 $(\text{pk}_0, \text{sk}_0) \leftarrow \text{KEM}_0.\text{KeyGen}()$	08 if $c = c^*$
02 $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KEM}_1.\text{KeyGen}()$	09 return $\perp$
03 $(k_0^*, c_0^*) \leftarrow \text{KEM}_0.\text{Encap}(\text{pk}_0)$	10 $k_0 \leftarrow \text{KEM}_0.\text{Decap}(\text{sk}_0, c_0)$
04 $(k_1^*, c_1^*) \leftarrow \text{KEM}_1.\text{Encap}(\text{pk}_1)$	11 $k_1 \leftarrow \text{KEM}_1.\text{Decap}(\text{sk}_1, c_1)$
05 $k^* \leftarrow \mathcal{K}$	12 if $k_0 = \perp$ or $k_1 = \perp$
06 $b' \leftarrow \mathcal{A}^{\text{DO}_{\text{Decap}}}((\text{pk}_0, \text{pk}_1), (c_0^*, c_1^*), k^*)$	13 return $\perp$
07 return b'	14 if $c_0 \neq c_0^*$ and $k_0 = k_0^*$ and $c_1 = c_1^*$ // G_5
	15 abort // G_5
	16 $k \leftarrow \mathbf{H}(k_0, k_1, \text{algotlabel})$
	17 return k

Fig. 10: Games $G_5 - G_6$ for the proof of Theorem 14.

Game G_6 . As a final change, we undo the remaining deviation from $\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 1)$ —i.e., we remove the abort that happens whenever the honest shared key, $k^* = \mathbf{H}(k_0^*, k_1^*, \text{algotlabel})$, could be given to the attacker through a permitted decapsulation query (as described in G_1). With this, everything is exactly as in the case $\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 1)$, meaning that

$$\Pr[G_6(\mathcal{A}) = 1] = \Pr[\text{IND-CCA}_{\text{KEM}}(\mathcal{A}, 1) = 1] .$$

Again, the game only differs from its predecessor if $\mathcal{A}$ performed a successful C2PRI attack on KEM_0 . We turn this into a successful C2PRI reduction $\mathcal{B}_2$. Since C2PRI attackers on KEM_0 get the secret key used by KEM_0 and can generate the key pair for KEM_1 on their own, $\mathcal{B}_2$ can perfectly

simulate the decapsulation oracle and immediately recognize and output any C2PRI break (c_0) that might appear in $\mathcal{A}$'s decapsulation queries. From this, it follows that

$$|\Pr[G_5(\mathcal{A}) = 1] - \Pr[G_6(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}(\mathcal{B}_2) .$$

Finally, using the triangle inequality, we can derive

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) &\leq 2 \cdot \text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}(\mathcal{B}) + 2 \cdot \text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}(\mathcal{C}) \\ &\quad + 2 \cdot \text{Adv}_{\text{H}}^{\text{skPRF}}(\mathcal{D}) + 2\delta , \end{aligned}$$

where we folded $\mathcal{B}_1$ and $\mathcal{B}_2$ into a single C2PRI adversary $\mathcal{B}$, $\mathcal{C}_1$ and $\mathcal{C}_2$ into a single IND-CCA adversary $\mathcal{C}$, and $\mathcal{D}_1$ and $\mathcal{D}_2$ into a single skPRF adversary $\mathcal{D}$. $\square$

3.2 Security: Binding

Next, we analyze the binding properties of QSI and QSF, focusing mainly on the former, when instantiated with *generic* KEMs—the consequences of instantiating the frameworks with FO-based KEMs is covered afterward. Given the extensive range of binding properties, we present a selection here in the interest of space, deferring the rest to Section D. Specifically, in this section, we examine all binding properties within the LEAK adversarial model. Since the analyses are somewhat similar across other models, this selection should be decently representative of the entire range while also (1) allowing for a concise exposition and (2) demonstrating results in a relatively strong model.

QSI. Loosely speaking, the results we establish for QSI differ along two dimensions. The first dimension concerns the use of a collision-resistant hash function, or simply any function, for key derivation. The second dimension concerns the *kind of preservation* QSI achieves for the considered property. Here, we distinguish between two kinds of preservation that we refer to as *existential* and *universal* preservation. For the former, QSI possesses a property if *at least one* constituent KEM possesses a property; for the latter, QSI possesses a property if *every* constituent KEM possesses a property. In essence, this means that QSI is at least as secure as its strongest constituent KEM for the properties it existentially preserves. Contrarily, for properties it universally preserves, QSI merely ensures it does not “undo” a property possessed by every constituent KEM. All obtained results hold in both the classical and quantum setting, though the bounds may vary slightly due to the distinct concrete collision probabilities of the random oracle used in each context. Table 2 provides a high-level overview of the results, including those in the HON and MAL models.

Remark 15. The results involving a collision-resistant hash function can be made more concrete by modeling the hash function as a random oracle. In this setting, the CR advantage can be replaced by the corresponding random-oracle collision bounds, expressed in terms of the number of oracle queries made by the reduction adversary. By now, these bounds are well established to be (on the order of) $q^s/|\mathcal{K}|$, where q denotes the number of queries, with $s = 2$ in the (classical) ROM and $s = 3$ in the QROM [Zha15]. Adapting the proofs to this oracle setting is straightforward.

In total, there are six properties to go over:¹⁵ LEAK-BIND-K-PK, K,CT-PK, K-CT, K,PK-CT, CT-K, and CT-PK. The statements and proofs overlap considerably—in fact, some are merely special cases of others. As such, we cover several of them at once or cross-reference whenever appropriate.

Theorem 16 (QSI Universal Preservation of LEAK-BIND-K-CT and LEAK-BIND-K,PK-CT). *Let KEM_0 and KEM_1 be KEMs with respective key spaces $\mathcal{K}_0$ and $\mathcal{K}_1$, $\text{algotlabel} \in \mathcal{L}$ an algorithm-specific label, and $\text{H} : \mathcal{K}_0 \times \mathcal{K}_1 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSI}[\text{KEM}_0, \text{KEM}_1, \text{H}]$.*

¹⁵ Recall that we do not consider properties with the public key as *sole* binding source.

Table 2: High-level overview of the established results on the binding properties of QSI and QSF. For QSI (first section), the first row indicates how QSI satisfies the binding properties specified by each column: $\checkmark^\forall$ denotes universal preservation; $\checkmark^\exists$ denotes existential preservation. The remaining rows indicate the component properties that appear in the corresponding security bounds, excluding the preserved binding properties of constituent KEMs. ϵ_{pk} denotes the probability that two independent key-generation executions produce the same public key. For existential preservation, KEM_b refers to the constituent KEM for which the binding property is preserved. For QSF (second section), the table uses a similar structure and notation, but the result types differ: $\checkmark$ denotes satisfaction (through preservation or more generally), χ^* denotes non-preservation (while satisfaction may still hold), and χ denotes non-satisfaction. Accordingly, for QSF we explicitly indicate the relevant properties of the constituent KEM.

	X-BIND ($X \in \{\text{HON}, \text{LEAK}\}$)				MAL-BIND	
	K-CT K,PK-CT	K-PK K,CT-PK	CT-K	CT-PK	K-CT; K,PK-CT K-PK; K,CT-PK	CT-K CT-PK
Result (QSI)	$\checkmark^\forall$	$\checkmark^\exists$	$\checkmark^\forall$	$\checkmark^\exists$	$\checkmark^\forall$	$\checkmark^\forall$
KEM_b	—	ϵ_{pk}	—	ϵ_{pk}	—	—
KEM_{1-b}	—	—	—	—	—	—
H	CR	CR	—	—	CR	—
Result (QSF)	$\checkmark$	$\checkmark$	χ^*	$\checkmark$	$\checkmark$	χ
KEM_0	K-CT K,PK-CT	—	—	CT-PK and ϵ_{pk}	K-CT; K,PK-CT K-PK; K,CT-PK	—
$\mathcal{N}$	—	ϵ_{pk}	—	—	—	—
H	CR	CR	—	—	CR	—

Then, for any adversary $\mathcal{A}$ against $\text{LEAK-BIND-P-Q} \in \{\text{LEAK-BIND-K-CT}, \text{LEAK-BIND-K,PK-CT}\}$ for KEM (as defined in Fig. 4), it holds that

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-P-Q}}(\mathcal{A}) &\leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-P-Q}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\text{KEM}_1}^{\text{LEAK-BIND-P-Q}}(\mathcal{D}) , \end{aligned}$$

where adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$, $\mathcal{C} := \mathcal{C}(\mathcal{A})$, $\mathcal{D} := \mathcal{D}(\mathcal{A})$ are as constructed in the proof of this theorem.

Proof. For adversary $\mathcal{A}$ to succeed in the LEAK-BIND-K-CT game for KEM, it must provide two *different* encapsulations $c_0 := (c_{00}, c_{01}) \neq (c_{10}, c_{11}) =: c_1$ that decapsulate to the *same* shared keys $k_0 = k_1$ under decapsulation keys $\text{sk}_0 := (\text{sk}_{00}, \text{sk}_{01})$ and $\text{sk}_1 := (\text{sk}_{10}, \text{sk}_{11})$ that *may or may not* be equal (as $\mathcal{A}$ may decide itself whether to consider the same or independently generated key pairs). Indeed, in this game, the final $\text{BB}_{\text{P,Q}}$ predicate reduces to $k_0 = k_1 \neq \perp \wedge c_0 \neq c_1$. Per the decapsulation procedure of KEM, this means that

$$\begin{aligned} &\text{H}(\text{KEM}_0.\text{Decap}(\text{sk}_{00}, c_{00}), \text{KEM}_1.\text{Decap}(\text{sk}_{01}, c_{01}), \text{algotlabel}) \\ &= \\ &\text{H}(\text{KEM}_0.\text{Decap}(\text{sk}_{10}, c_{10}), \text{KEM}_1.\text{Decap}(\text{sk}_{11}, c_{11}), \text{algotlabel}) . \end{aligned}$$

Now, if any of the inputs to H on the left-hand side is unequal to its counterpart on the right-hand side, this implies a collision for H. So, as a first step, we split the probability space based on an event

D that captures the case where *any* corresponding inputs are unequal, and bound accordingly by constructing an adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ against CR of H. That is,

$$\begin{aligned} & \text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-K-CT}}(\mathcal{A}) \\ &= \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1] \\ &= \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] + \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge D] \\ &\leq \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] + \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) . \end{aligned}$$

It remains to bound the case where *both* corresponding inputs are equal (i.e., $\neg D$). To this end, observe that $c_{00} \neq c_{10} \vee c_{01} \neq c_{11}$ (because $c_0 \neq c_1$). As such, we split the probability space again, this time on the event $C := c_{00} \neq c_{10}$; note that, at this point, $\neg C$ implies $c_{01} \neq c_{11}$. That is,

$$\begin{aligned} \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] &= \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge C] \\ &\quad + \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge \neg C] . \end{aligned}$$

Considering the former (where C holds), we simultaneously have $c_{00} \neq c_{10}$ and $\text{KEM}_0.\text{Decap}(c_{00}, \text{sk}_0) = \text{KEM}_0.\text{Decap}(c_{10}, \text{sk}_0)$. Indeed, since the LEAK-BIND-K-CT game for KEM encompasses the analogous game for KEM_0 —essentially executing it in parallel with that for KEM_1 —(c_0, c_1) constitute a LEAK-BIND-K-CT break for KEM_0 . Hence, we can construct an adversary $\mathcal{C} := \mathcal{C}(\mathcal{A})$ against LEAK-BIND-K-CT for KEM_0 that, first, runs $\mathcal{A}_1$ and directly returns its output, indicating whether to consider the same or independently generated key pairs. Then, upon receiving the (potentially duplicated) key pairs from its own game, $\mathcal{C}$

1. uses KEM_1 to generate either one or two key pairs, depending on the choice previously made by $\mathcal{A}_1$;
2. combines the resulting key pairs with the ones it received from its own game, perfectly simulating the key pairs expected by $\mathcal{A}_2$; and
3. runs $\mathcal{A}_2$ on the final result.

Throughout, $\mathcal{C}$ directly forwards any queries $\mathcal{A}$ issues to H, trivially simulating this aspect of $\mathcal{A}$'s view. After $\mathcal{A}_2$ returns (c_0, c_1) , $\mathcal{C}$ extracts and returns (c_{00}, c_{10}) . Certainly, per the considered case, $\mathcal{C}$ succeeds in its own game and, hence,

$$\Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge C] \leq \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}_0}(\mathcal{C}) = 1] .$$

The other case (where $\neg C$ holds), is completely symmetrical, merely substituting any artifact from KEM_0 by the analogous one from KEM_1 . As a result, we can similarly construct an adversary $\mathcal{D} := \mathcal{D}(\mathcal{A})$ such that

$$\Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge \neg C] \leq \Pr[\text{LEAK-BIND-K-CT}_{\text{KEM}_1}(\mathcal{D}) = 1] .$$

By definition, the above probability statements involving $\mathcal{C}$ and $\mathcal{D}$ are the advantages on the right-hand side of the inequality in the theorem statement. Consequently, we obtain the desired bound by combining the individual steps.

For LEAK-BIND-K,PK-CT, a simplified version of the proof applies. Here, the primary difference is that $\mathcal{A}$ *does not* get to decide whether to consider the same or independently generated key pairs (nor do $\mathcal{C}$ and $\mathcal{D}$); instead, the game always considers a single (duplicated) key pair. Following, it suffices to construct $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ similarly to before, yet omitting any logic accounting for ($\mathcal{A}$'s choice of) independently generated key pairs. Then, since the overall key pair for KEM is the same, each of its components are (pair-wise) the same as well, particularly implying that the contained key pairs for KEM_0 and KEM_1 are respectively the same (required for $\mathcal{C}$ and $\mathcal{D}$ to succeed in their own game). $\square$

Theorem 17 (QSI Existential Preservation of LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK). *Consider the same setting as for Theorem 16. Then, for any adversary $\mathcal{A}$ against LEAK-BIND-P-Q $\in \{\text{LEAK-BIND-K-PK}, \text{LEAK-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_b}^{\text{LEAK-BIND-P-Q}}(\mathcal{C}_b) + \epsilon_b ,$$

for $b \in \{0, 1\}$. Here, $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C}_b := \mathcal{C}_b(\mathcal{A})$ are as constructed in the proof of this theorem, and ϵ_b denotes the probability of KEM_b generating the same public key in two independent executions of its key generation procedure.

Proof. For LEAK-BIND-K-PK, $\mathcal{A}$ succeeds if it provides *any* two encapsulations $c_0 := (c_{00}, c_{01})$ and $c_1 := (c_{10}, c_{11})$ that successfully decapsulate to the *same* shared key $k_0 = k_1$ under two independently generated decapsulation keys $\text{sk}_0 := (\text{sk}_{00}, \text{sk}_{01})$ and $\text{sk}_1 := (\text{sk}_{10}, \text{sk}_{11})$. In this game, the $\text{BB}_{\text{P,Q}}$ predicate reduces to $k_0 = k_1 \neq \perp \wedge \text{pk}_0 \neq \text{pk}_1$. Using similar reasoning (and notation) as in the proof of Theorem 16, it follows that

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-K-PK}}(\mathcal{A}) \leq \Pr[\text{LEAK-BIND-K-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] + \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) .$$

Here, $\neg D$ represents the case where *both* corresponding inputs to H —i.e., the decapsulations from KEM_0 and KEM_1 , as internally performed by KEM —are equal. Certainly, barring the pathological case where independently generated key pairs contain the same public key, this implies that (c_{00}, c_{10}) and (c_{01}, c_{11}) constitute LEAK-BIND-K-PK breaks for KEM_0 and KEM_1 , respectively. More formally, we can split the probability space on an event P that captures the case where two key pairs, independently generated by KEM_0 , contain the same public key, and derive

$$\Pr[\text{LEAK-BIND-K-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] \leq \Pr[\text{LEAK-BIND-K-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge \neg P] + \epsilon_0 ,$$

At this point, the remaining probability statement concerns a case where $\mathcal{A}$'s output contains a LEAK-BIND-K-PK break for KEM_0 . As such, we construct an adversary $\mathcal{C}_0 := \mathcal{C}_0(\mathcal{A})$ that perfectly simulates $\mathcal{A}$'s game, integrating the key pairs it receives from its own game. By the considered case, $\mathcal{C}_0$ is guaranteed to succeed by extracting and returning (c_{00}, c_{10}) from $\mathcal{A}$'s output. That is,

$$\Pr[\text{LEAK-BIND-K-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge \neg P] \leq \Pr[\text{LEAK-BIND-K-PK}_{\text{KEM}_0}(\mathcal{C}_0) = 1] .$$

Merging the individual steps, we obtain

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-K-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-K-PK}}(\mathcal{C}_0) + \epsilon_0 .$$

Furthermore, an analogous analysis shows that we can similarly construct an adversary $\mathcal{C}_1 := \mathcal{C}_1(\mathcal{A})$ such that

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-K-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_1}^{\text{LEAK-BIND-K-PK}}(\mathcal{C}_1) + \epsilon_1 .$$

Combining the final two inequalities gives the desired result.

For LEAK-BIND-K,CT-PK, an identical proof applies. Indeed, the only difference is that the encapsulations c_0 and c_1 (and thus c_{00} and c_{10} , as well as c_{01} and c_{11}) are guaranteed to be equal, which does not affect the reasoning. $\square$

Theorem 18 (QSI Universal Preservation of LEAK-BIND-CT-K). *Consider the same setting as for Theorem 16, but let H be any function. Then, for any adversary $\mathcal{A}$ against LEAK-BIND-CT-K for KEM (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-CT-K}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-CT-K}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_1}^{\text{LEAK-BIND-CT-K}}(\mathcal{C}) .$$

Here, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem.

Proof. For LEAK-BIND-CT-K, $\mathcal{A}$ succeeds if it provides two *equal* encapsulations $c_0 := (c_{00}, c_{01}) = (c_{10}, c_{11}) =: c_1$ that successfully decapsulate to two *different* shared keys $k_0 \neq k_1$ under decapsulation keys $\text{sk}_0 := (\text{sk}_{00}, \text{sk}_{01})$ and $\text{sk}_1 := (\text{sk}_{10}, \text{sk}_{11})$ that *may or may not* be equal. In this game, the $\text{BB}_{P,Q}$ predicate reduces to $c_0 = c_1 \wedge \perp \neq k_0 \neq k_1 \neq \perp$. Similarly to before, it follows (per the decapsulation procedure of KEM) that

$$\begin{aligned} & \text{H}(\text{KEM}_0.\text{Decap}(\text{sk}_{00}, c_{00}), \text{KEM}_1.\text{Decap}(\text{sk}_{01}, c_{01}), \text{algotlabel}) \\ & \quad \neq \\ & \text{H}(\text{KEM}_0.\text{Decap}(\text{sk}_{10}, c_{10}), \text{KEM}_1.\text{Decap}(\text{sk}_{11}, c_{11}), \text{algotlabel}) . \end{aligned}$$

In turn, it must be the case that *at least one* of the inputs to H on the left-hand side is unequal to its counterpart on the right-hand side. Thus, we split the probability space on the event $E := \text{KEM}_0.\text{Decap}(c_{00}, \text{sk}_{00}) \neq \text{KEM}_0.\text{Decap}(c_{10}, \text{sk}_{10})$, noting that $\neg E$ implies $\text{KEM}_1.\text{Decap}(c_{01}, \text{sk}_{01}) \neq \text{KEM}_1.\text{Decap}(c_{11}, \text{sk}_{11})$ at this point. Then, in the former case, (c_{00}, c_{10}) represents a LEAK-BIND-CT-K break for KEM_0 : c_{00} and c_{10} are *equal* and decapsulate to *different* shared keys. Thus, we construct an adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ that perfectly simulates $\mathcal{A}$'s game while integrating the key pairs it receives from its own game. By the considered case, extracting and returning (c_{00}, c_{10}) from $\mathcal{A}$'s final output ensures that $\mathcal{B}$ succeeds. That is,

$$\Pr[\text{LEAK-BIND-CT-K}_{\text{KEM}}(\mathcal{A}) = 1 \wedge E] \leq \Pr[\text{LEAK-BIND-CT-K}_{\text{KEM}_0}(\mathcal{B}) = 1] .$$

The other case (where $\neg E$ holds), is entirely analogous; particularly, we can similarly construct an adversary $\mathcal{C} := \mathcal{C}(\mathcal{A})$ for which

$$\Pr[\text{LEAK-BIND-CT-K}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg E] \leq \Pr[\text{LEAK-BIND-CT-K}_{\text{KEM}_1}(\mathcal{C}) = 1] .$$

By definition, the probability statements involving $\mathcal{B}$ and $\mathcal{C}$ are equal to the relevant advantages. Combining the individual steps gives the desired result. $\square$

Theorem 19 (QSI Existential Preservation of LEAK-BIND-CT-PK). *Consider the same setting as for Theorem 16, but let H be any function. Then, for any adversary $\mathcal{A}$ against LEAK-BIND-CT-PK for KEM (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_b}^{\text{LEAK-BIND-CT-PK}}(\mathcal{B}_b) + \epsilon_b ,$$

for $b \in \{0, 1\}$. Here, $\mathcal{B}_b := \mathcal{B}_b(\mathcal{A})$ is as constructed in the proof of this theorem, and ϵ_b denotes the probability of KEM_b generating the same public key in two independent executions of its key generation procedure.

Proof. For LEAK-BIND-CT-PK, $\mathcal{A}$ succeeds if it provides two *equal* encapsulations $c_0 := (c_{00}, c_{01}) = (c_{10}, c_{11}) =: c_1$ that successfully decapsulate to *any* shared keys k_0 and k_1 under two independently generated decapsulation keys $\text{sk}_0 := (\text{sk}_{00}, \text{sk}_{01})$ and $\text{sk}_1 := (\text{sk}_{10}, \text{sk}_{11})$. In this game, the $\text{BB}_{P,Q}$ predicate reduces to $c_0 = c_1 \wedge k_0 \neq \perp \wedge k_1 \neq \perp \wedge \text{pk}_0 \neq \text{pk}_1$. Observe that KEM's decapsulation runs the decapsulation procedures of both KEM_0 and KEM_1 and, crucially, fails as soon as *at least one* of them fails. Excluding the pathological case of independently generated public keys being equal, (c_{00}, c_{10}) and (c_{01}, c_{11}) are LEAK-BIND-CT-PK breaks for KEM_0 and KEM_1 , respectively, if (c_0, c_1) is a LEAK-BIND-CT-PK break for KEM. So, proceeding as in the proof for Theorem 17 (and reusing notation), we derive

$$\Pr[\text{LEAK-BIND-CT-PK}_{\text{KEM}}(\mathcal{A}) = 1] \leq \Pr[\text{LEAK-BIND-CT-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg P] + \epsilon_0 ,$$

where $\neg P$ represents the case where two key pairs, independently generated by KEM_0 , contain *different* public keys. Now, the probability on the right-hand side of the inequality concerns a case where $\mathcal{A}$'s output contains a LEAK-BIND-CT-PK break for KEM_0 . Thus, we construct an adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ that perfectly simulates $\mathcal{A}$'s game while integrating the key pairs it receives from its own game. Once $\mathcal{A}$ returns, $\mathcal{B}_0$ extracts and returns (c_{00}, c_{10}) which, per the considered case, guarantees $\mathcal{B}_0$'s success:

$$\Pr[\text{LEAK-BIND-CT-PK}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg P] \leq \Pr[\text{LEAK-BIND-CT-PK}_{\text{KEM}_0}(\mathcal{B}_0) = 1] .$$

Combining the preceding steps, we obtain

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-CT-PK}}(\mathcal{B}_0) + \epsilon_0 .$$

Analogously, we can construct an adversary $\mathcal{B}_1 := \mathcal{B}_1(\mathcal{A})$ such that

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_1}^{\text{LEAK-BIND-CT-PK}}(\mathcal{B}_1) + \epsilon_1 .$$

Together, the last two inequalities yield the desired result. $\square$

QSF. Following the discussion for **QSI**, we consider all binding properties within the **LEAK** adversarial model for **QSF** as well. Recall that **QSF** considers a **KEM** and a nominal group (instead of two **KEMs**); furthermore, **QSF** includes the shared key, encapsulation, and public key from the nominal group in the derivation of its own shared key (see Fig. 12). Consequently, **QSF** satisfies certain binding properties without requiring any binding-like properties from the nominal group, provided the employed hash function is modeled as a random oracle. Further, some of **QSF**'s binding properties can be established independently of the underlying **KEMs**—the bounds only contain terms capturing (1) the generation of the same public key in two independent key generations for the nominal group and (2) a collision occurring in the key derivation function. Thus, in the theorems for **QSF**'s binding properties, there is no mention of universal or existential preservation: Either it preserves the binding property of its *only* constituent **KEM**, or it satisfies the binding property itself without relying on any security property of the underlying **KEM** and nominal group (not including the distinctness of independently generated public keys for the nominal group). Table 2 provides an overview of the results, including those in the **HON** and **MAL** models.

In the proofs of the theorems, we reuse notation and omit reasoning steps that are similar to ones covered before to prevent unnecessary repetition, occasionally referencing the appropriate proof steps instead.

Theorem 20 (QSF Preservation of LEAK-BIND-K-CT and LEAK-BIND-K,PK-CT). *Let KEM_0 be a **KEM** with key space $\mathcal{K}_0$, $\mathcal{N}$ a nominal group, $\text{algolabel} \in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{G}^3 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSF}[\text{KEM}_0, \mathcal{N}, H]$. Then, for any adversary $\mathcal{A}$ against $\text{LEAK-BIND-P-Q} \in \{\text{LEAK-BIND-K-CT}, \text{LEAK-BIND-K,PK-CT}\}$ for **KEM** (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_H^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-P-Q}}(\mathcal{C}) ,$$

where adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem.

Proof. A **LEAK-BIND-K-CT** break for **KEM** consists of a single encapsulation that decapsulates to two different shared keys. By the decapsulation procedure of **KEM**, this means that

$$\begin{aligned} & H(\text{KEM}_0.\text{Decap}(\text{sk}_{00}, c_{00}), k_{01}, c_{01}, \text{pk}_{01}, \text{algolabel}) \\ &= \\ & H(\text{KEM}_0.\text{Decap}(\text{sk}_{10}, c_{10}), k_{11}, c_{11}, \text{pk}_{11}, \text{algolabel}) , \end{aligned}$$

where k_{b1} , c_{b1} , and pk_{b1} are the shared keys, encapsulations, and public keys for the nominal group, respectively. As in earlier proofs, splitting the probability space on an event D that captures a collision for H gives rise to the CR advantage term for adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$, merely leaving a case in which all corresponding inputs to H are equal. Particularly, *both* $\text{KEM}_0.\text{Decap}(c_{00}, \text{sk}_{00}) = \text{KEM}_0.\text{Decap}(c_{10}, \text{sk}_{10})$ and $c_{01} = c_{11}$; since $(c_{00}, c_{01}) \neq (c_{10}, c_{11})$ (from the fact that $\mathcal{A}$ returns a LEAK-BIND-K-CT break), the latter implies $c_{00} \neq c_{10}$. Hence, (c_{00}, c_{10}) constitutes a LEAK-BIND-K-CT break for KEM_0 . As such, we can construct an adversary $\mathcal{C} := \mathcal{C}(\mathcal{A})$ nearly identical to the one constructed in the LEAK-BIND-K-CT proof for QSI (Theorem 16); in fact, the differences are mostly syntactical, simply accounting for the differences in handled artifacts. This adversary perfectly simulates the appropriate environment for $\mathcal{A}$ while integrating the values it receives from its own game, succeeding whenever $\mathcal{A}$ would have. This leads to a bound on the final case which, combined with the previous bound, yields the desired result.

For LEAK-BIND-K,PK-CT, an analogous analysis with a simpler reduction adversaries ($\mathcal{B}$ and $\mathcal{C}$) applies. (Here, the game invariably considers the same key pair and, hence, $\mathcal{B}$ and $\mathcal{C}$ do not have to account for any decision from $\mathcal{A}$ regarding the considered key pairs.) $\square$

Theorem 21 (QSF LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK). *Consider the same setting as for Theorem 20. Then, for any adversary $\mathcal{A}$ against $\text{LEAK-BIND-P-Q} \in \{\text{LEAK-BIND-K-PK}, \text{LEAK-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \epsilon_{\mathcal{N}} ,$$

where $\mathcal{B} := \mathcal{B}(\mathcal{A})$ is as constructed in the proof of this theorem, and $\epsilon_{\mathcal{N}}$ denotes the probability of KEM generating the same public key for $\mathcal{N}$ in two independent executions of its key generation procedure.

Proof. A LEAK-BIND-K-PK break for KEM comprises two encapsulations that decapsulate to the same shared keys under two independently generated decapsulation keys. Akin to the other proofs with the shared key as binding source, we carve out and bound the case where a collision for H occurs, leaving the case where all inputs to the oracle are equal. Now, since the public keys of the nominal group are included in the input to H , they are equal per this remaining case; further, they are independently generated per the assumption that $\mathcal{A}$ returned a LEAK-BIND-K-PK break. Therefore, we can bound the probability of the remaining case by $\epsilon_{\mathcal{N}}$. Combining the established bounds, the result follows.

As the above reasoning does not depend on the (in)equality of the provided encapsulations, an identical proof applies to LEAK-BIND-K,CT-PK. $\square$

Theorem 22 (QSF Preservation of LEAK-BIND-CT-PK). *Consider the same setting as for Theorem 20, but let H be any function. Then, for any adversary $\mathcal{A}$ against LEAK-BIND-CT-PK for KEM (as defined in Fig. 4), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{LEAK-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{LEAK-BIND-CT-PK}}(\mathcal{B}) + \epsilon_0 .$$

Here, adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ is as constructed in the proof of this theorem, and ϵ_0 denotes the probability of KEM_0 generating the same public key in two independent executions of its key generation procedure.

Proof. A LEAK-BIND-CT-PK break for KEM consists of a single encapsulations that decapsulates to any (valid) shared keys under two independently generated decapsulation keys. In particular, this implies that KEM internally runs the decapsulation procedure of KEM_0 on the same encapsulation using two independently generated decapsulation keys. Crucially, both of these runs succeed and return a

valid shared key since, if any of them failed, KEM would fail as well. Consequently, as long as the key pairs (independently) generated for KEM_0 have distinct public keys, a LEAK-BIND-CT-PK break for KEM contains an analogous break for KEM_0 . Following similar reasoning as the proof for Theorem 19, we bound the case where two public keys, independently generated by KEM_0 , are equal by ϵ_0 . Then, we construct an adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ such that its advantage against LEAK-BIND-CT-PK for KEM_0 bounds the remaining case. From this, the desired result follows. $\square$

Finally, QSF does *not* (necessarily) preserve LEAK-BIND-CT-K. Breaking this property requires a *single* encapsulation that decapsulates to *distinct* shared keys. If the key pairs are generated independently, then distinct public keys of the nominal group already suffice for the resulting shared keys to differ—barring a collision in the key-derivation function—provided that the component encapsulation decapsulates successfully under both secret keys. But this only amounts to a LEAK-BIND-CT-PK break for the component KEM and need not be a LEAK-BIND-CT-K break there, since the component shared keys may still be equal.

3.3 Plugging in FO-Based KEMs

Following the analysis of QSI (and QSF) for fully generic constituent KEMs, we explore where FO-based KEMs fit in. That is, considering the results we established for completely generic KEMs, we aim to demonstrate in what sense the different variants of the FO transform guarantee the C2PRI, IND-CCA, and binding properties—this (indirectly) shows which of the variants may be used to instantiate the frameworks securely. The IND-CCA security of the different variants has already been analyzed previously, e.g., in [HHK17] and many follow-up works. Also, the binding properties of the different variants have recently been analyzed in [KSW25a, KMSW26]. Consequently, we focus on the C2PRI property in this work.

More precisely, we show that the proof of [BCD⁺24, Theorem 3, p. 16]—capturing C2PRI for ML-KEM-768—can be generalized to any KEM obtained by applying a variant of the U-transform to a rigid, deterministic PKE scheme. In fact, for some cases, we do not even require rigidity. However, in any case, the results apply to KEMs constructed via (a variant of) the full FO transform, since the T-transform produces a rigid, deterministic PKE scheme (from *any* PKE scheme).

In the ensuing, we first establish the most general result for $U_m^\perp$, specifically when F and H are implemented by the same function (modeled as a random oracle). Afterward, we consider the case where F and H are independent random oracles—or at least explicitly domain-separated. Then, we proceed to $U^\perp$, the implicitly rejecting variant that additionally hashes the encapsulation. We also briefly go over the findings for the explicitly rejecting variants, $U_m^\perp$ and $U^\perp$, which are mostly implied by those for their implicitly rejecting counterparts. In short, the analyses for $U_m^\perp$ and $U^\perp$ are almost identical to those for $U_m^\perp$ and $U^\perp$, but simply disregard one of the cases (as the adversary cannot win sending a failing encapsulation). Finally, we discuss further variations used in practice, particularly salted FO and FO with key confirmation.

Theorem 23. *Let dPKE be a deterministic, rigid PKE scheme and $H : \mathcal{M} \rightarrow \mathcal{K}$ a random oracle with F as alias, i.e., $F := H$.¹⁶ Further, let $\text{KEM} := U_m^\perp[\text{dPKE}, H, F]$. Then, for any adversary $\mathcal{A}$ against C2PRI for KEM (as defined in Fig. 3), issuing at most $q \geq 0$ classical queries to H , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{1}{|\mathcal{K}_F|} + \frac{q+2}{|\mathcal{K}|}.$$

¹⁶ According to the definitions of the U-transforms (see Definition 12), H and F do not (necessarily) have the same domains. However, for some results in this section, we implicitly assume the existence of a consistent encoding of elements from these different domains into a single joint domain $\mathcal{X}$ for the random oracles (e.g., via bitstrings). This way, statements such as $F := H$ and $H = F$ make sense.

If $\mathcal{A}$ may issue quantum queries, it instead holds that

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{1}{|\mathcal{K}_F|} + \frac{8(2q+3)^2}{|\mathcal{K}|}.$$

Proof. On a high level, the proof is a three-step argument. First, we show that an adversary that can break C2PRI is able to find a collision for an externally provided value. Then, we argue that this value is uniform in a sufficiently large subset of the domain. Lastly, we present a reduction from generalized second-preimage finding (see Definition 32), bounding the probability of actually finding such a collision.

Step 1. Assume $\mathcal{A}$ provides a valid C2PRI break $c_{\mathcal{A}}$. This means that *both*

$$\text{KEM.Decap}(\text{sk}, c_{\mathcal{A}}) = k^* \tag{1}$$

and

$$c_{\mathcal{A}} \neq c^* = \text{Enc}(\text{pk}, m^*) . \tag{2}$$

Note that we always have $k^* = H(m^*)$ for some m^* sampled in Encap as it is defined as the partial output of an honest encapsulation. There are two cases to consider (referring to the decryption performed during the decapsulation of $c_{\mathcal{A}}$): $c_{\mathcal{A}}$ decrypts successfully, i.e., it is a valid ciphertext, or not.

Case 1: $c_{\mathcal{A}}$ valid. If $c_{\mathcal{A}}$ is a valid ciphertext, decryption succeeds and results in a message $m_{\mathcal{A}} = \text{Dec}(\text{sk}, c_{\mathcal{A}})$. By the rigidity of dPKE, we have that $c_{\mathcal{A}} = \text{Enc}(\text{pk}, m_{\mathcal{A}})$. In addition, given that dPKE is a *deterministic* encryption scheme and Eq. (2) holds, it follows that $m^* \neq m_{\mathcal{A}}$ —otherwise, the same message would encrypt to c^* and $c_{\mathcal{A}}$. Hence, we have $H(m^*) = H(m_{\mathcal{A}})$ and $m^* \neq m_{\mathcal{A}}$, while m^* was not chosen by the adversary.

Case 2: $c_{\mathcal{A}}$ invalid. If $c_{\mathcal{A}}$ is an invalid ciphertext, decryption fails, meaning decapsulation returns $F(\text{fk}, c_{\mathcal{A}})$. Thus, $F(\text{fk}, c_{\mathcal{A}}) = H(\text{fk}, c_{\mathcal{A}}) = H(m^*)$. Here, we again distinguish two cases. First, if the inputs are of different length, they are trivially distinct; since m^* is provided externally, the claim follows. Otherwise (i.e., the inputs are of the same length), the probability of the inputs being equal is upper-bounded by $\frac{1}{|\mathcal{K}_F|}$: fk is chosen uniformly at random from $\mathcal{K}_F$, and equality of the overall input particularly requires equality between fk and the corresponding part of m^* (which is uniformly random due to m^* being uniformly random as a whole). Hence, with probability at least $1 - \frac{1}{|\mathcal{K}_F|}$, we can derive a second preimage for some externally given value from the adversary's output.

Step 2. In all of the above cases, the externally given part of the collision is m^* , which is sampled from the uniform distribution over $\mathcal{M}$.

Step 3. We can give a direct reduction from GSPR (where the parameter space is empty) of $H = F$ for challenges sampled from $\mathcal{M}$. The reduction, given a challenge input $x \leftarrow^s \mathcal{M}$ simply sets $m^* \leftarrow x$ during the generation of the C2PRI challenge, observes $\mathcal{A}$'s queries, and, depending on the case, returns $m_{\mathcal{A}}$ or $(\text{fk}, c_{\mathcal{A}})$. Applying Lemma 35 and collecting the terms then leads to the claimed bounds. $\square$

Then, for the other case—i.e., where H and F are independent random oracles—we establish the following result.

Theorem 24. *Let dPKE be a deterministic, rigid PKE scheme, and $H : \mathcal{M} \rightarrow \mathcal{K}$ and $F : \mathcal{K}_F \times \mathcal{C} \rightarrow \mathcal{K}$ (independent) random oracles. Further, let $\text{KEM} := \text{U}_m^{\mathcal{K}}[\text{dPKE}, H, F]$. Then, for any adversary $\mathcal{A}$ against C2PRI for KEM (as defined in Fig. 3), respectively issuing at most $q_H \geq 0$ and $q_F \geq 0$ classical queries to H and F , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{q_H + q_F + 3}{|\mathcal{K}|}.$$

If $\mathcal{A}$ may issue quantum queries, it instead holds that

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{8((2q_H + 3)^2 + (2q_F + 1)^2)}{|\mathcal{K}|}.$$

Proof. The proof is identical to the above for **Case 1**. If we are in this case, we can again apply the reduction from GSPR of H and Lemma 35.

However, if the adversary outputs c_A that falls into **Case 2**, the situation is slightly different. In this case, $\mathcal{A}$ outputs c_A such that $F(\text{fk}, c_A) = H(m^*) = k^*$, where k^* is externally given. In this case, we can give a reduction from OW for F : Given a challenge y for F , the reduction replaces H by the random oracle $H' := H_{m^* \mapsto y}$ that maps m^* to y and replaces k^* by y in the input to $\mathcal{A}$. Given that y is a random value, H' is a random function and the joint distribution of $(H, k^*) = (H', y)$ that $\mathcal{A}$ has access to is identical to that in the real game. Any C2PRI solution c_A returned by $\mathcal{A}$ leads to a valid preimage (fk, c_A) for y under F . Applying Lemma 34 and a union bound gives the claimed bounds. $\square$

Next, we proceed to the other implicitly rejecting variant, $U^\perp$, which additionally uses the encapsulation as part of the input to H . For this variant, we show that the above bounds hold; in fact, in the former case, the bound improves.

Theorem 25. *Theorems 23 and 24 hold when instantiating KEM as $U^\perp[\text{dPKE}, H, F]$. Further, in the setting of Theorem 23 (i.e., when $H = F$), the bounds improve to*

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{q + 2}{|\mathcal{K}|}$$

when $\mathcal{A}$ issues classical queries, and

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{8(2q + 3)^2}{|\mathcal{K}|}$$

when $\mathcal{A}$ issues quantum queries.

Proof. Here, the only difference to the settings of Theorems 23 and 24 concerns k , as it gets set to $H(m, c)$ instead of $H(m)$. Considering the same case distinction as before, for **Case 1**, this means that $H(m^*, c^*) = H(m_A, c_A)$. Here, we are guaranteed by the game that $c^* \neq c_A$ and, hence, $(m^*, c^*) \neq (m_A, c_A)$ —i.e., the adversary found a second preimage for H . Additionally, $c^* = \text{Enc}(\text{pk}, m^*)$, which means that the externally given target value is uniform in the set $(m, \text{Enc}(\text{pk}, m))$. In this case, we can construct a reduction from generalized second preimage finding for H where the parameter space is given by the key-pair space of KEM, equipped with the distribution induced by its key generation algorithm. For a sampled key pair (pk, sk) , the parameterized input space then consists of elements of the desired form: $(m, \text{Enc}(\text{pk}, m))$. The resulting GSPR reduction is identical to the ones in Theorems 23 and 24, except that it uses the key pair provided by its own game rather than generating it internally, and returns the pair (m_A, c_A) of just m_A . Given that H is a random function, this does not change the bound compared to before.

For **Case 2**, nothing changes in the analysis for the setting $H \neq F$, except that we program $(m^*, c^*) \mapsto y$ rather than just m^* . The remaining analysis is limited to F , which is identical. For the setting where $H = F$, the argument simplifies. We have $F(\text{fk}, c_A) = H(\text{fk}, c_A) = H(m^*, c^*)$. Again, if the inputs are of different length, they are trivially distinct; since (m^*, c^*) is externally given, the claim is satisfied. If the inputs are of the same length, they are different by the winning condition $c_A \neq c^*$. Hence, the bound improves as we can drop the $\frac{1}{|\mathcal{K}_F|}$ term. $\square$

Remark 26. At the cost of adding an additive term δ (the correctness error) to the bounds, the above also holds without the requirement for rigidity.

So far, we have covered the *implicitly* rejecting variants $U^\perp$ and $U_m^\perp$. The analysis simplifies if we turn to the *explicitly* rejecting variants $U^\perp$ and $U_m^\perp$. For these variants, we get a single results that covers both because they inherently only use a single key derivation function.

Theorem 27. *Let dPKE be a deterministic, rigid PKE scheme, and H a random oracle. Further, let $KEM := U_m^\perp[dPKE, H]$. Then, for any adversary $\mathcal{A}$ against C2PRI for KEM (as defined in Fig. 3), issuing at most $q \geq 0$ classical queries to H, it holds that*

$$\text{Adv}_{KEM}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{q+2}{|\mathcal{K}|}.$$

If $\mathcal{A}$ may issue quantum queries, it instead holds that

$$\text{Adv}_{KEM}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{8(2q+3)^2}{|\mathcal{K}|}.$$

The bounds are identical when $KEM := U^\perp[dPKE, H]$.

Proof. The proof immediately follows from the analysis above noticing that Case 2 can never occur. This is because the adversary cannot succeed by sending an invalid ciphertext that gets rejected, since $k^* \neq \perp$ by construction (encapsulation cannot fail). $\square$

While this covers all variants of the conventional FO transform, many real-world constructions use variations of FO that do not directly fall into this category. In the following, we discuss two special such cases: salted FO, used by HQC and FrodoKEM, and FO with key confirmation, used by (variants of) Classic McEliece and (streamlined) NTRU Prime.

SALTED FO. The salted FO transform (SFO) was introduced by the FrodoKEM team in an Annex to their latest specification [ABD⁺23] and later analyzed in [GHS25] with the goal of mitigating the impact of multi-user and -challenge attacks. Later, it was also adopted by HQC. The change to FO, i.e., from those shown in Figs. 5 and 6, is that a value **salt** is sampled uniformly at random as part of the encryption of the T-transform. This value becomes an input to the randomness generation via G (which, in this case, additionally takes the public key, but that is irrelevant for C2PRI). Further, it becomes part of the produced ciphertext. As such, the salt increases the number of possible ciphertexts. Afterward, they apply a variant of the $U^\perp$ (FrodoKEM) or $U_m^\perp$ (HQC) transform where the salt becomes an additional input to the functions that previously took the PKE ciphertext c as input or derived shared/rejection keys.¹⁷ FrodoKEM thereby feeds the c , **salt**, and public key into the computation of both the shared and rejection key, while HQC leaves out the c in case of the shared (non-rejection) key.

Proposition 28. *For the implicitly rejecting variants of the SFO transform, the C2PRI bounds are identical to those of their non-salted counterparts, as given by Theorems 24 and 25.*

Proof (Sketch). In this case the analysis for Theorem 25 fully applies as it essentially only is based on the fact that either computation takes the ciphertext as input in the same position. As the ciphertexts

¹⁷ FrodoKEM still performs the two stage hashing, where the final session key is the output of a key-derivation that takes the ciphertext (thereby also **salt**), and a prekey k that was produced as a function of the public key, the message, and **salt**. While this complicates the IND-CCA proof [MX23, BH23], it is irrelevant for the C2PRI analysis.

are different by assumption, a solution to C2PRI implies a generalized second preimage for H . In this case, the generalized second preimage challenge is sampled from a slightly larger (parameterized) input space than in Theorem 25 as it also includes the salt. $\square$

KEY CONFIRMATION. The first analysis of FO in the QROM [TU16] added a (almost length-preserving) hash of the message, called a key-confirmation hash, to the encapsulation to enable a security proof.¹⁸ When more advanced proof techniques rendered the additional hash obsolete, it was omitted by all proposals considered in this work, except for (streamlined) NTRU Prime and Classic McEliece (in the version considered by ISO, at the time of writing). In short, a key-confirmation variant for any of the considered FO variants is obtained by attaching $H'(m)$ to the encapsulation, for some dedicated hash function $H' \neq H$.¹⁹ During decapsulation, the key-confirmation hash is recomputed and compared to the value included in the encapsulation. In case of a mismatch, the encapsulation gets rejected. It is important to note that the key-confirmation hash also becomes an input of the key confirmation. derivations, we only consider the case where F and H are modeled as independent random oracles.

Proposition 29. *For U^χ with key confirmation and H and F (independent) random oracles, the C2PRI bounds are identical to those of its counterpart without key confirmation, as given by Theorem 25.*

Proof (Sketch). The analysis is analogous to that of Theorem 25. $\square$

4 QSF & QSI with Real-World KEMs

As a final consideration, we examine the usability of the most relevant post-quantum KEMs with the QSF and QSI combiners. As demonstrated in Section 3, the fundamental requirements for such usability—that is, for the resulting combiner KEM to achieve IND-CCA—are IND-CCA and C2PRI security. Further, regarding the binding properties, the requirements heavily depend on the expected security of the resulting combiner KEM. As a general rule of thumb, though, if the resulting KEM should satisfy property X-BIND-P-Q, the constituent KEM(s) should also satisfy X-BIND-P-Q.

Since all KEMs under consideration have already been analyzed with respect to IND-CCA, we focus on the C2PRI property in what follows, leaving the analysis of the binding properties for future work. Table 3 summarizes the obtained bounds on C2PRI (advantage) for the considered KEMs.

4.1 ML-KEM

The C2PRI security of ML-KEM (as specified in FIPS 203 [oST24]) has already been analyzed in [BCD⁺24]. Although the analysis in [BCD⁺24] focuses explicitly on ML-KEM-768, the same proof and bound apply to the other parameter sets, as they only depend on the functions $G := \text{SHA3-512}$ and $J := \text{SHAKE256}(\cdot, 256)$, which are globally fixed (i.e., independent of the particular parameter set). Observing that ML-KEM internally constructs a rigid, deterministic PKE scheme and subsequently applies U_m^χ with $H := \text{trunc}(G, 256)$ —i.e., the truncation of G ’s output to the first 256 bits—and $F := J$, the application of Theorem 24 gives the bounds listed in Table 3 (where we let $q = q_H + q_F$).

Importantly, these bounds assume SHA3-512 and SHAKE256 can securely be modeled as (quantum) random oracles up to q_H and q_F (quantum) queries, respectively. For the classical setting, the

¹⁸ This was necessary to extract messages from ciphertexts by simulating the random oracle using an invertible function (specifically, a $2q$ -wise independent hash function).

¹⁹ Not reusing H is crucial as to not leak the shared key [BDG20].

Table 3: Overview of the obtained bounds on C2PRI (advantage) for the considered KEMs, in both the classical and quantum settings. Here, q denotes an upper bound on the number of evaluations of the key derivation function(s) by the adversary. For clarity, the bounds exclude some terms whose contribution would be irrelevant for any practical value for q .

KEM	Bound on C2PRI	
	Classical	Quantum
ML-KEM	$(q + 3)/2^{256}$	$8(2q + 3)^2/2^{256}$
HQC	$(q + 3)/2^{256}$	$8(2q + 3)^2/2^{256}$
(e)FrodoKEM-640	$(q + 2)/2^{128}$	$8(2q + 3)^2/2^{128}$
(e)FrodoKEM-976	$(q + 2)/2^{192}$	$8(2q + 3)^2/2^{192}$
(e)FrodoKEM-1344	$(q + 2)/2^{256}$	$8(2q + 3)^2/2^{256}$
Classic McEliece	$(q + 1)/2^{256}$	$8(2q + 1)^2/2^{256}$
sntrup	$(q + 2)/2^{256}$	$8(2q + 3)^2/2^{256}$
NTRU-HPs	$(q + 2)/2^{256}$	$(8(2q + 3)^2 + 1)/2^{256}$
NTRU-HRSS	$(q + 2)/2^{256}$	$(8(2q + 3)^2 + 1)/2^{256}$

indifferentiability of the sponge construction—the construction underlying SHAKE and SHA3—has been proven in the random permutation model up to $2^{\frac{c}{2}}$ queries [BDPV08], where the capacity c equals 512 for SHAKE256 and $2X$ for SHA3- X . This justifies the assumption to the point where the bounds become trivial/irrelevant. For the quantum setting, the indifferentiability of the sponge construction has successfully evaded analysis for quite some time. A recent preprint [ACMT25] claims a successful analysis against quantum adversaries, but proves a rather loose bound. For the above bounds to remain meaningful, we only require indifferentiability up to $2^{\frac{c}{4}}$ queries, which is still more than [ACMT25] achieves. Nevertheless, this bound seems like a reasonable conjecture because the exact bound is related to finding collisions in the capacity-related part of the internal state: In the quantum setting, collision finding requires *at least* $2^{\frac{c}{4}}$ queries, at the cost of $2^{\frac{c}{2}}$ storage [BHT98].

4.2 HQC

NIST selected HQC [GAMA⁺25] for standardization as part of the fourth round of its PQC standardization process [ABC⁺25]. The construction of HQC can be captured by the salted FO variant based on $U_m^\mathcal{L}$ with independent hash functions for H and F. Concretely, the proposed instantiation of H and F are SHA3-512 truncated to the first 256 bits (capacity of 1024) and SHA3-256 (capacity of 512), respectively. Accordingly, Proposition 28 (invoking Theorem 24) yields the relevant C2PRI bounds for HQC. Since both the C2PRI bounds and the capacities of the employed functions coincide with those for ML-KEM, we estimate the same overall bounds, as reflected in Table 3.

4.3 (e)FrodoKEM

(e)FrodoKEM [NAB⁺20] builds upon the $U^\mathcal{L}$ transform. Specifically, the recent version (FrodoKEM) uses the SFO transform; the original version (eFrodoKEM) uses the regular (non-salted) transform [ABD⁺23]. (e)FrodoKEM is currently under consideration for standardization in ISO/IEC [fro25]. Depending on the targeted security level, (e)FrodoKEM instantiates the function H of its transform differently: (e)FrodoKEM-640 uses SHAKE128($\cdot$, 128), (e)FrodoKEM-976 uses SHAKE256($\cdot$, 192), and (e)FrodoKEM-1344 uses SHAKE256($\cdot$, 256).

The previous discussion concerning the indistinguishability of SHAKE similarly applies here. In particular, this means that—under the same assumptions and conjectures as before—Theorem 25 and Proposition 28 give rise to the bounds listed in Table 3.

4.4 Classic McEliece

Currently, Classic McEliece [ABC⁺22] is under consideration for standardization in ISO/IEC [cla25] in two variants, both of which are built from a rigid, deterministic PKE scheme and use a variation of the $U^\perp$ transform. However, they differ in the specific variation for the latter: one variant uses the (regular) $U^\perp$ transform, the other uses the $U^\perp$ transform with *key confirmation* (as described in Section 3.3). In either case, the functions H and F are both instantiated using SHAKE256($\cdot$, 256), yet with explicit domain separation. As such, combining Theorem 25, Proposition 29, and an analysis (with corresponding assumptions) for SHAKE256 analogous to the ones above, the bounds listed in Table 3 follow.

4.5 NTRU

Several NTRU variants are currently under consideration for standardization by the IETF/IRTF: NTRU Prime [BBC⁺20] in its streamlined form, `sntrup`, for SSH [FMJ25]; and both NTRU-HPS and NTRU-HRSS [CDH⁺20] for IKEv2 [FKS⁺25, FPC⁺25].

`sntrup` involves a rigid, deterministic PKE scheme [Ber22] and uses the $U^\perp$ transform with key confirmation. Further, it uses SHA2-512—truncated to the first 32 bytes—to instantiate both H and F. As with Classic McEliece, however, explicit domain separation is enforced. Thus, assuming this truncated version of SHA2-512 can be modeled as a random oracle up to 2^{256} classical queries and up to 2^{128} quantum queries,²⁰ Proposition 29 bounds the C2PRI advantage for `sntrup`, leading to the values reported in Table 3.

Both, NTRU-HPS and NTRU-HRSS, apply the $U_m^\perp$ transform to a rigid, deterministic PKE scheme, instantiating both H and F with SHA3-256, using no domain separation. Hence, reasoning along similar lines to before, Theorem 23 gives rise to the respective bounds listed in Table 3.²¹

4.6 Interpretation of Bounds

For all schemes except (e)FrodoKEM, the C2PRI bounds in Table 3 are governed by the 256-bit hash choices and are therefore uniform across parameter sets. (e)FrodoKEM is the exception: its bounds scale with the selected 128/192/256-bit hash instantiation, where those hash lengths are chosen to match the intended security level of (e)FrodoKEM-640, (e)FrodoKEM-976, (e)FrodoKEM-1344, respectively.

²⁰ For the classical setting this is justified by [CDMP05], observing that a fixed input length is enforced by the construction here. For the quantum setting, the proven bound is $q^3/\sqrt{2^n}$ [Zha19] (where $n = 512$ in this case), which is worse than what we require. At the same time, though, the proof for this bound is based on the same observation as the one in the classical setting: the best way of distinguishing a Merkle-Damgård hash function (with fixed length inputs) from a random oracle is to find a collision in its internal state. This would take at least $q^3/2^n$ for any quantum adversary. Assuming the truth lies somewhere in between, say at most at $q^4/2^n$, justifies the assumption in the quantum setting as well.

²¹ Notice that, in this case, $|\mathcal{K}_F| = |\mathcal{K}| = |\{0, 1\}^{256}|$.

References

- ABC⁺22. Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- ABC⁺25. Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David A. Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the fourth round of the NIST Post-Quantum Cryptography standardization process. NIST Interagency/Internal Report (NISTIR) 8545, National Institute of Standards and Technology (NIST), March 2025. doi:10.6028/NIST.IR.8545.
- ABD⁺23. Erdem Alkim, Joppe W. Bos, Léo Ducas, Patrick Longa, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Chris Peikert, Ananth Raghunathan, and Douglas Stebila. Annex on FrodoKEM updates, April 2023. URL: <https://frodokem.org/files/FrodoKEM-annex-20230418.pdf>.
- ABH⁺21. Joël Alwen, Bruno Blanchet, Eduard Hauck, Eike Kiltz, Benjamin Lipp, and Doreen Riepel. Analysing the HPKE standard. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 87–116. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_4.
- ABK25. Gorjan Alagic, Fahren Bajaj, and Aybars Kocoglu. The best of both KEMs: Securely combining KEMs in post-quantum hybrid schemes. Cryptology ePrint Archive, Paper 2025/1444, 2025. URL: <https://eprint.iacr.org/2025/1444>.
- ACMT25. Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The Sponge is quantum indifferentiable. Cryptology ePrint Archive, Report 2025/731, 2025. URL: <https://eprint.iacr.org/2025/731>.
- AS04. Scott Aaronson and Yaoyun Shi. Quantum lower bounds for the collision and the element distinctness problems. *J. ACM*, 51(4):595–605, July 2004. doi:10.1145/1008731.1008735.
- BBC⁺20. Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, Chitchanok Chuengsatiansup, Tanja Lange, Adrian Marotzke, Bo-Yuan Peng, Nicola Tuveri, Christine van Vredendaal, and Bo-Yin Yang. NTRU Prime. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- BCD⁺24. Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karoline Varner, and Bas Westerbaan. X-Wing. *CiC*, 1(1):21, 2024. doi:10.62056/a3qj89n4e.
- BDG20. Mihir Bellare, Hannah Davis, and Felix Günther. Separate your domains: NIST PQC KEMs, oracle cloning and read-only indistinguishability. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 3–32. Springer, Cham, May 2020. doi:10.1007/978-3-030-45724-2_1.
- BDPV08. Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indistinguishability of the Sponge construction. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 181–197. Springer, Berlin, Heidelberg, April 2008. doi:10.1007/978-3-540-78967-3_11.
- Ber22. Daniel J. Bernstein. A one-time single-bit fault leaks all previous NTRU-HRSS session keys to a chosen-ciphertext attack. In Takanori Isobe and Santanu Sarkar, editors, *INDOCRYPT 2022*, volume 13774 of *LNCS*, pages 617–643. Springer, Cham, December 2022. doi:10.1007/978-3-031-22912-1_27.
- BH23. Manuel Barbosa and Andreas Hülsing. The security of Kyber’s FO-transform. Cryptology ePrint Archive, Report 2023/755, 2023. URL: <https://eprint.iacr.org/2023/755>.
- BHT98. Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In Claudio L. Lucchesi and Arnaldo V. Moura, editors, *LATIN 1998*, volume 1380 of *LNCS*, pages 163–169. Springer, Berlin, Heidelberg, April 1998. doi:10.1007/bfb0054319.

- BP18. Daniel J. Bernstein and Edoardo Persichetti. Towards KEM unification. Cryptology ePrint Archive, Report 2018/526, 2018. URL: <https://eprint.iacr.org/2018/526>.
- Can25. Canadian Centre for Cyber Security. Roadmap for the migration to post-quantum cryptography for the Government of Canada. Technical Report ITSM.40.001, Communications Security Establishment Canada, June 2025. URL: <https://www.cyber.gc.ca/en/guidance/roadmap-migration-post-quantum-cryptography-government-canada-itsm40001>.
- CB25. Deirdre Connolly and Richard Barnes. Concrete hybrid PQ/T key encapsulation mechanisms. Technical report, Internet Engineering Task Force, July 2025. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-concrete-hybrid-kems/00/>.
- CBG25. Deirdre Connolly, Richard Barnes, and Paul Grubbs. Hybrid PQ/T key encapsulation mechanisms. Technical report, Internet Engineering Task Force, July 2025. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-hybrid-kems/05/>.
- CDH⁺20. Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hulsing, Joost Rijneveld, John M. Schanck, Peter Schwabe, William Whyte, Zhenfei Zhang, Tsunekazu Saito, Takashi Yamakawa, and Keita Xagawa. NTRU. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- CDM24. Cas Cremers, Alexander Dax, and Niklas Medinger. Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 1046–1060. ACM Press, October 2024. doi:10.1145/3658644.3670283.
- CDMP05. Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 430–448. Springer, Berlin, Heidelberg, August 2005. doi:10.1007/11535218_26.
- cla25. Classic McEliece. Preliminary Standardization Proposal ISO/IEC 18033-2 DAM 2, International Organization for Standardization, 2025. URL: <https://www.iso.org/standard/86890.html>.
- Com25. European Commission. A coordinated implementation roadmap for the transition to post-quantum cryptography, June 2025. URL: <https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography>.
- CS03. Ronald Cramer and Victor Shoup. Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM Journal on Computing*, 33(1):167–226, 2003.
- DHK⁺21. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, and Gregor Seiler. Faster lattice-based KEMs via a generic Fujisaki-Okamoto transform using prefix hashing. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2722–2737. ACM Press, November 2021. doi:10.1145/3460120.3484819.
- DKKW25. Justin Drake, Dmitry Khovratovich, Mikhail A. Kudinov, and Benedikt Wagner. Hash-based multi-signatures for post-quantum ethereum. *CiC*, 2(1):13, 2025. doi:10.62056/ae7qjpp10.
- ETS25. ETSI Technical Committee CYBER. ETSI Technical Specification 103 744 V1.2.1: Quantum-safe hybrid key establishment. Technical report, ETSI, March 2025. URL: https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.02.01_60/ts_103744v010201p.pdf.
- FKS⁺25. Yuta Fukagawa, Hisaharu Kosuge, Masataka Saito, Scott Fluhrer, and Akira Nagai. Post-quantum hybrid key exchange with NTRU in the Internet Key Exchange Protocol Version 2 (IKEv2). Technical report, Internet Engineering Task Force, July 2025. URL: <https://datatracker.ietf.org/doc/draft-skyline-ipsecme-ntru-ikev2/00/>.
- FLP14. Marc Fischlin, Anja Lehmann, and Krzysztof Pietrzak. Robust multi-property combiners for hash functions. *Journal of Cryptology*, 27(3):397–428, July 2014. doi:10.1007/s00145-013-9148-7.
- FMJ25. Markus Friedl, Jan Mojžiš, and Simon Josefsson. Secure Shell (SSH) key exchange method using hybrid streamlined NTRU Prime sntrup761 and X25519 with SHA-512. Technical report, Internet Engineering Task Force, July 2025. URL: <https://datatracker.ietf.org/doc/draft-ietf-sshm-ntruprime-ssh/04>.

- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013. doi:10.1007/s00145-011-9114-1.
- FPC⁺25. Scott Fluhrer, Michael Prorock, Sofia Celi, John Gray, Xagawa Keita, and Haruhisa Kosuge. NTRU key encapsulation. Technical report, Internet Engineering Task Force, July 2025. URL: <https://datatracker.ietf.org/doc/draft-fluhrer-cfrg-ntru/03/>.
- fro25. FrodoKEM: Learning with errors key encapsulation. Preliminary Standardization Proposal ISO/IEC 18033-2 DAM 2, International Organization for Standardization, 2025. URL: <https://www.iso.org/standard/86890.html>.
- GAMA⁺25. Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuvill, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jérôme Lacan, Jean-Marc Robert, Pascal Véron, Paulo L. Barreto, Santosh Ghosh, Shay Gueron, Tim Güneysu, Rafael Misoczki, Jan Richter-Brokmann, Nicolas Sendrier, Jean-Pierre Tillich, and Valentin Vasseur. Hamming Quasi-Cyclic (HQC). Technical report, August 2025. URL: https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf.
- GHP18. Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 190–218. Springer, Cham, March 2018. doi:10.1007/978-3-319-76578-5_7.
- GHS25. Lewis Glabush, Kathrin Hövelmanns, and Douglas Stebila. Tight multi-challenge security reductions for key encapsulation mechanisms. *Cryptology ePrint Archive*, Report 2025/343, 2025. URL: <https://eprint.iacr.org/2025/343>.
- Her05. Amir Herzberg. On tolerant cryptographic constructions. In Alfred Menezes, editor, *CT-RSA 2005*, volume 3376 of *LNCS*, pages 172–190. Springer, Berlin, Heidelberg, February 2005. doi:10.1007/978-3-540-30574-3_13.
- HHK17. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017. doi:10.1007/978-3-319-70500-2_12.
- HRS16. Andreas Hülsing, Joost Rijneveld, and Fang Song. Mitigating multi-target attacks in hash-based signatures. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part I*, volume 9614 of *LNCS*, pages 387–416. Springer, Berlin, Heidelberg, March 2016. doi:10.1007/978-3-662-49384-7_15.
- Jos25. Simon Josefsson. Chempat: Generic instantiated PQ/T hybrid key encapsulation mechanisms. Technical report, Internet Engineering Task Force, March 2025. URL: <https://datatracker.ietf.org/doc/draft-josefsson-chempat/03/>.
- KMSW26. Juliane Krämer, Yannick Münz, Patrick Struck, and Maximiliane Weishäupl. Binding security of explicitly-rejecting KEMs via plaintext confirmation and robust PKEs. *IACR Communications in Cryptology*, 2(4), 2026. doi:10.62056/anudvurz.6.
- KSW25a. Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Binding security of implicitly-rejecting KEMs and application to BIKE and HQC. *IACR Communications in Cryptology*, 2(2), 2025. doi:10.62056/ak2i893y6.
- KSW25b. Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. A note on the binding properties of KEM combiners. *Cryptology ePrint Archive*, Paper 2025/1416, 2025. URL: <https://eprint.iacr.org/2025/1416>.
- ML10. Johannes Merkle and Manfred Lochter. Elliptic curve cryptography (ECC) Brainpool standard curves and curve generation. RFC 5639, March 2010. URL: <https://www.rfc-editor.org/info/rfc5639>, doi:10.17487/RFC5639.
- MX23. Varun Maram and Keita Xagawa. Post-quantum anonymity of Kyber. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 3–35. Springer, Cham, May 2023. doi:10.1007/978-3-031-31368-4_1.
- NAB⁺20. Michael Naehrig, Erdem Alkim, Joppe Bos, Léo Ducas, Karen Easterbrook, Brian LaMacchia, Patrick Longa, Ilya Mironov, Valeria Nikolaenko, Christopher Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.

- Nat25a. National Cyber Security Centre. Timelines for migration to post-quantum cryptography. Technical report, National Cyber Security Centre (NCSC), GCHQ, March 2025. URL: <https://www.ncsc.gov.uk/guidance/pqc-migration-timelines>.
- Nat25b. National Security Agency. Announcing the Commercial National Security Algorithm Suite 2.0. Cybersecurity Advisory PP-22-1338, U/OO/194427-22, National Security Agency, May 2025. URL: https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGO_RITHMS.PDF.
- oST23. National Institute of Standards and Technology. Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters. Technical Report NIST SP 800-186, U.S. Department of Commerce, February 2023. URL: <https://doi.org/10.6028/NIST.SP.800-186>.
- oST24. National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, U.S. Department of Commerce, August 2024. URL: <https://doi.org/10.6028/NIST.FIPS.203>.
- Sch24. Sophie Schmieg. Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK. *IACR Cryptol. ePrint Arch.*, page 523, 2024.
- TU16. Ehsan Ebrahimi Targhi and Dominique Unruh. Post-quantum security of the Fujisaki-Okamoto and OAEP transforms. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part II*, volume 9986 of *LNCS*, pages 192–216. Springer, Berlin, Heidelberg, October / November 2016. doi:10.1007/978-3-662-53644-5_8.
- Zha15. Mark Zhandry. A note on the quantum collision and set equality problems. *Quantum Info. Comput.*, 15(7–8):557–567, May 2015.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019. doi:10.1007/978-3-030-26951-7_9.

A Definitions

For completeness, this appendix presents the formal definitions, as well as some corresponding results, omitted from the main body for clarity of exposition.

A.1 Hash Functions

The hash function properties considered throughout (collision resistance, one-wayness, and (generalized) second-preimage resistance) are defined below.

Definition 30 (Collision Resistance (CR)). *Given a hash function $H : \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ against the collision resistance of H is defined as*

$$\text{Adv}_H^{\text{CR}}(\mathcal{A}) := \Pr[H(x) = H(x') \mid (x, x') \leftarrow \mathcal{A}()] .$$

Definition 31 (One-Wayness (OW)). *Given a hash function $H : \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ against the one-wayness of H is defined as*

$$\text{Adv}_H^{\text{OW}}(\mathcal{A}) := \Pr[H(x) = y \mid y \leftarrow \mathcal{Y}; x \leftarrow \mathcal{A}(y)] .$$

Definition 32 (Generalized Second-Preimage Resistance (GSPR)). *Let $\mathcal{X}$ be an input space that may be restricted by parameters from parameters space $\mathcal{P}$, denoted $\mathcal{X}_p$ (for $p \in \mathcal{P}$). Then, for a hash function $H : \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ against the generalized second-preimage resistance of H is defined as*

$$\text{Adv}_{H, \mathcal{P}}^{\text{GSPR}}(\mathcal{A}) := \Pr \left[H(x) = H(x') \wedge x \neq x' \mid \begin{array}{l} p \leftarrow \mathcal{P}; \\ x \leftarrow \mathcal{X}_p; \\ x' \leftarrow \mathcal{A}(x, p) \end{array} \right] .$$

The following lemmas state concrete bounds on each of the above-defined properties whenever the hash function is modeled as a random oracle. We provide proof sketches for all but the collision resistance bound, for which literature is abundant—e.g., see [Zha15, AS04].

Lemma 33. *Let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Then, for any adversary $\mathcal{A}$ against CR of H , issuing at most q_H classical queries to H , it holds that*

$$\text{Adv}_H^{\text{CR}}(\mathcal{A}) \leq \frac{q_H^2 + 1}{|\mathcal{Y}|}.$$

If $\mathcal{A}$ may issue quantum queries, it instead holds

$$\text{Adv}_H^{\text{CR}}(\mathcal{A}) \leq \frac{q_H^3 + 1}{|\mathcal{Y}|}.$$

Lemma 34. *Let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Then, for any adversary $\mathcal{A}$ against OW of H , issuing at most q_H classical queries to H , it holds that*

$$\text{Adv}_H^{\text{OW}}(\mathcal{A}) \leq \frac{q_H + 1}{|\mathcal{Y}|}.$$

If $\mathcal{A}$ may issue quantum queries, it instead holds

$$\text{Adv}_H^{\text{OW}}(\mathcal{A}) \leq \frac{8(2q_H + 1)^2}{|\mathcal{Y}|}.$$

Proof (Sketch). For classical queries, every query has probability $|\mathcal{Y}|^{-1}$ of hitting y . If no match was found with q queries, $\mathcal{A}$ can return a final guess.

When the adversary is given quantum access, the bound can be proven via a reduction from a search problem over boolean functions. See for example [DKKW25] where the claimed bound is proven for the case of tweakable hash functions (that take a tweak and a public parameter as additional inputs). As it is independent of the size of tweak space and public parameter space, it also applies to functions with empty tweak and public parameter space, covering the case of a plain hash function. Our bound has an additional factor 2 which is a correction that comes from tracing constants through their proof and noticing that we have to uncompute query results, which also requires a query, causing the number to double. $\square$

Lemma 35. *Let $\mathcal{P}$ be a parameter space and $\mathcal{X}$ an input space that may be restricted by parameters from $\mathcal{P}$. Further, let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Then, for any adversary $\mathcal{A}$ against GSPR of H , issuing at most q_H classical queries to H , it holds that*

$$\text{Adv}_{H,\mathcal{P}}^{\text{GSPR}}(\mathcal{A}) \leq \frac{q_H + 1}{|\mathcal{Y}|}.$$

If $\mathcal{A}$ may issue quantum queries, it instead holds

$$\text{Adv}_{H,\mathcal{P}}^{\text{GSPR}}(\mathcal{A}) \leq \frac{8(2q_H + 1)^2}{|\mathcal{Y}|}.$$

Proof (Sketch). First, note that this property is essentially second-preimage finding where the challenge is not chosen uniformly at random from the whole domain of the function but from a subset. Further, for random oracles, remark that the challenge could even simply be a fixed value, as the success probability of the adversary is taken over the random coins of the oracle. As such, the bounds follow from those of regular second-preimage finding. These have, e.g., been discussed and proven in [HRS16]. Noticing that the proofs therein apply even for a fixed challenge x , the desired result follows. $\square$

Evidently, the above proof (sketch) relies on the fact that, for a random oracle, the second-preimage challenge could be fixed. However, for actual instantiations, it is important that the challenge actually comes from a sufficiently large set to exclude for example precomputation attacks. While we consider the analysis of bounds for specific hash functions out of scope, we think that this detailed analysis can be helpful for a potential later analysis.

A.2 Key Encapsulation Mechanisms

For KEMs, the definition for binding properties in the malicious model is given below.

Definition 36 (KEM Binding (MAL) [CDM24]). Let $P, Q \subset \{\text{PK}, \text{C}, \text{K}\}$ such that $P \cap Q = \emptyset$, where $\text{PK} := \{\text{pk}_0, \text{pk}_1\}$, $\text{C} := \{c_0, c_1\}$, and $\text{K} := \{k_0, k_1\}$. Further, define “binding break” predicate

$$\begin{aligned} \text{BB}_{P,Q} : \mathcal{PK} \times \mathcal{PK} \times \mathcal{C} \times \mathcal{C} \times \mathcal{K} \times \mathcal{K} &\rightarrow \{\text{true}, \text{false}\} \\ \text{BB}_{P,Q}(\text{pk}_0, \text{pk}_1, c_0, c_1, k_0, k_1) &:= \\ k_0 \neq \perp \wedge k_1 \neq \perp \wedge \forall_{p \in P} : p_0 = p_1 \wedge \exists_{q \in Q} : q_0 \neq q_1 &, \end{aligned}$$

where p_b (resp. q_b) refers to the respective element in the set p (resp. q); e.g., if $p = \text{PK}$, p_0 denotes pk_0 . Lastly, let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be a KEM for which the function $\text{sktopk} : \mathcal{SK} \rightarrow \mathcal{PK}$ maps secret keys to public keys in line with KeyGen . Then, given an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ playing in the MAL-BIND-P-Q game (as defined in Fig. 15), the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{KEM}}^{\text{MAL-BIND-P-Q}}(\mathcal{A}) := \Pr[\text{MAL-BIND-P-Q}_{\text{KEM}}(\mathcal{A}) = 1] .$$

Game MAL-BIND-P-Q_{KEM}($\mathcal{A}$): <pre> 01 $d \leftarrow \mathcal{A}_1()$ 02 if $d = \text{DecapDecap}$ 03 $\text{sk}_0, \text{sk}_1, c_0, c_1 \leftarrow \mathcal{A}_2()$ 04 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 05 $k_0 \leftarrow \text{Decap}(\text{sk}_0, c_0)$ 06 $k_1 \leftarrow \text{Decap}(\text{sk}_1, c_1)$ 07 else if $d = \text{EncapDecap}$ 08 $\text{sk}_0, \text{sk}_1, r_0, c_1 \leftarrow \mathcal{A}_2()$ 09 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 10 $k_0, c_0 \leftarrow \text{Encap}(\text{pk}_0; r_0)$ 11 $k_1 \leftarrow \text{Decap}(\text{sk}_1, c_1)$ 12 else // $d = \text{EncapEncap}$ 13 $\text{sk}_0, \text{sk}_1, r_0, r_1 \leftarrow \mathcal{A}_2()$ 14 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 15 $k_0, c_0 \leftarrow \text{Encap}(\text{pk}_0; r_0)$ 16 $k_1, c_1 \leftarrow \text{Encap}(\text{pk}_1; r_1)$ 17 return $\text{BB}_{P,Q}(\text{pk}_0, \text{pk}_1, c_0, c_1, k_0, k_1)$ </pre>

Fig. 11: MAL-BIND-P-Q game for $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ and $P, Q \subset \{\text{PK}, \text{C}, \text{K}\}$ such that $P \cap Q = \emptyset$.

KeyGen()	Encap(pk := (pk ₀ , pk ₁))	Decap(sk := (sk ₀ , sk ₁ , pk ₁), c := (c ₀ , c ₁))
01 (pk ₀ , sk ₀) ← KEM ₀ .KeyGen()	07 (k ₀ , c ₀) ← KEM ₀ .Encap(pk ₀)	14 k ₀ ← KEM ₀ .Decap(sk ₀ , c ₀)
02 sk ₁ ← _{\$} ε _h	08 sk _e ← _{\$} ε _h	15 k ₁ ← c ₁ ^{sk₁}
03 pk ₁ ← g ^{sk₁}	09 c ₁ ← _{\$} g ^{sk_e}	16 if k ₀ = ⊥
04 pk ← (pk ₀ , pk ₁)	10 k ₁ ← pk ₁ ^{sk_e}	17 return ⊥
05 sk ← (sk ₀ , sk ₁ , pk ₁)	11 k ← H(k ₀ , k ₁ , c ₁ , pk ₁ , algolabel)	18 k ← H(k ₀ , k ₁ , c ₁ , pk ₁ , algolabel)
06 return (pk, sk)	12 c ← (c ₀ , c ₁)	19 return k
	13 return (k, c)	

Fig. 12: QSF[KEM₀, $\mathcal{N}$, H].

A.3 QSF

We reiterate the formal definition of nominal groups and the specification of QSF, both closely following the ones from the original paper proposing X-Wing (and QSF) [BCD⁺24]. For the definition of nominal groups, this paper in turn closely follows [ABH⁺21].

Definition 37 (Nominal Group [BCD⁺24, ABH⁺21]). A nominal group $\mathcal{N}$ is given by a tuple $(\mathcal{G}, g, p, \epsilon_h, \epsilon_u, \text{exp})$ where $\mathcal{G}$ is an efficiently recognizable finite set (of “group elements”), $g \in \mathcal{G}$ is a group element (referred to as “base element”), p is a prime, $\epsilon_h \subset \mathbb{Z}$ is a finite set (of “honest exponents”), $\epsilon_u \subset \mathbb{Z} \setminus p\mathbb{Z}$ is a finite set (of “exponents”), and $\text{exp} : \mathcal{G} \times \mathbb{Z} \rightarrow \mathcal{G}$ is an efficiently computable function (referred to as “exponentiation”) where, for $X \in \mathcal{G}$ and $y \in \mathbb{Z}$, X^y serves as a shorthand for $\text{exp}(X, y)$. Further, exp is such that the following holds:

1. For all $X \in \mathcal{G}$ and $y, z \in \mathbb{Z}$, $(X^y)^z = X^{yz}$.
2. The function $\phi(x) := g^x$ mapping ϵ_u to $\{g^x | x \in [1, p-1]\}$ is a bijection.

With the necessary definitions in place, Fig. 12 provides the specification of QSF.

B Hash Input Size and Performance Impact

In our framework, the final key is derived as

$$k = H(k_0, k_1, \text{algolabel}),$$

meaning that the hash input consists only of the two shared keys and a fixed domain-separation label. For typical parameter sets this corresponds to roughly 64–96 bytes.

Some combiner constructions additionally include KEM artifacts such as ciphertexts or public keys in the hash input. For example, QSF hashes values including c_1 and pk_1 . In the original setting of QSF these objects correspond to elliptic-curve group elements and therefore contribute only a few dozen bytes.

However, when such constructions are instantiated with post-quantum KEMs, the size of these artifacts can increase substantially. For instance, ML-KEM ciphertexts are 768–1568 bytes, while (e)FrodoKEM ciphertexts range from 9720 to 21632 bytes depending on the parameter set. Generic combiner constructions such as GHP also hash ciphertexts and public keys, and therefore exhibit the same behavior when instantiated with post-quantum KEMs.

Modern software implementations of SHA3-256 on commodity x86 CPUs typically achieve roughly 0.26–0.55 GB/s on long inputs. Under these assumptions, hashing an additional kilobyte of input adds roughly 1.8–3.8 μs , while hashing 10–22 kB adds on the order of 20–80 μs . Although this cost is small compared to the computational cost of post-quantum KEM operations themselves, it may become noticeable in latency-sensitive deployments.

By restricting the hash input to the shared keys, our framework keeps the hashing workload small and independent of the size of KEM artifacts.

KeyGen	Encap(pk := (pk ₀ , pk ₁))	Decap(sk := (sk ₀ , sk ₁), c := (c ₀ , c ₁))
01 (pk ₀ , sk ₀) ← KEM ₀ .KeyGen()	06 (k ₀ , c ₀) ← KEM ₀ .Encap(pk ₀)	11 k ₀ ← KEM ₀ .Decap(sk ₀ , c ₀)
02 (pk ₁ , sk ₁) ← KEM ₁ .KeyGen()	07 (k ₁ , c ₁) ← KEM ₁ .Encap(pk ₁)	12 k ₁ ← KEM ₀ .Decap(sk ₁ , c ₁)
03 sk ← (sk ₀ , sk ₁)	08 k ← H(k ₀ , k ₁ , c ₁ , algolabel)	13 if k ₀ = ⊥ or k ₁ = ⊥
04 pk ← (pk ₀ , pk ₁)	09 c ← (c ₀ , c ₁)	14 return ⊥
05 return (pk, sk)	10 return (k, c)	15 k ← H(k ₀ , k ₁ , c ₁ , algolabel)
		16 return k

Fig. 13: QSI variant that hashes c_1 into k .

C QSI: Variant and Multi-User/Challenge Security

C.1 Variant: Additionally Hashing an Encapsulation

We show that a c_1 -hashing variant of QSI achieves IND-CCA security already if KEM₁ satisfies a notion that is weaker than IND-CCA security. We formally define this QSI variant in Fig. 13.

We begin by defining the security notion in question, *one-wayness under plaintext-checking and validity attacks* (OW-PCVA) for KEMs. In essence, this notion is an abstraction of SDH for nominal group KEMs, capturing that it is hard to invert a KEM ciphertext. The plaintext-checking oracle can be viewed as an abstraction of the DH oracle, and the validity oracle captures the case where ciphertexts fail to decapsulate to a shared secret. (Which does not happen for group-based KEMs.)

Definition 38 (OW-PCVA). *Given a KEM $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$ and an adversary $\mathcal{A}$ playing in the OW-PCVA game (as defined in Fig. 14), the advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{\text{KEM}}^{\text{OW-PCVA}}(\mathcal{A}) = \Pr \left[\text{OW-PCVA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1 \right] .$$

Game OW-PCVA _{KEM} :	PCO(k, c)
01 (pk, sk) ← KeyGen()	05 return Decap(sk, c) = k
02 (k, c*) ← Encap(pk)	
03 k' ← A ^{PCO, VAL} (pk, c*)	VAL(c)
04 return k' = k	06 k ← Decap(sk, c)
	07 return k = ⊥

Fig. 14: OW-PCVA game for $\text{KEM} := (\text{KeyGen}, \text{Encap}, \text{Decap})$.

With the necessary definitions at hand, we now prove that the considered QSI variant (Fig. 13) is IND-CCA-secure if KEM₁ is OW-PCVA and KEM₀ is C2PRI-secure, hence weakening the requirement on one of the KEMs compared to the original QSI construction (c.f. Fig. 7 and Theorem 14).²²

Theorem 39. *let KEM₀ and KEM₁ be KEMs with respective key spaces $\mathcal{K}_0$ and $\mathcal{K}_1$, algolabel $\in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{K}_1 \times \mathcal{C} \times \mathcal{L} \rightarrow \mathcal{K}$ a random oracle. Further, let KEM be the instantiation of the QSI variant defined in Fig. 13 with KEM₀ and KEM₁. Then, for any adversary*

²² Note that, as opposed to Theorem 14, this proof is in the ROM; specifically, we model the key derivation function H as a random oracle.

$\mathcal{A}$ against IND-CCA of KEM (as defined in Fig. 2), respectively issuing at most $q_H \geq 0$ and $q_{DO} \geq 0$ to H and DO_{Decap} , it holds that

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_1}^{\text{OW-PCVA}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{C2PRI}}(\mathcal{C}) .$$

Here, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem. $\mathcal{B}$ issues at most q_H queries to its plaintext-checking oracle, and at most q_{DO} queries to its validity-checking oracle.

Proof. This proof loosely resembles the corresponding proof for QSF [BCD⁺24, Thm. 1]. We prove the statement via a sequence of games, starting with the original IND-CCA game for KEM as G_0 :

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \Pr[G_0(\mathcal{A}) = 1] .$$

Game G_1 . We first change the game such that it aborts whenever the challenge shared secret, $H(k_0^*, k_1^*, c_1^*, \text{algolabel})$, could be given to the attacker through a decapsulation query: concretely, G_1 aborts if $\mathcal{A}$ ever queries Decap on a ciphertext $(c_0, c_1) \neq (c_0^*, c_1^*)$ such that $\text{Decap}_0(c_0) = \text{Decap}_0(c_0^*)$.

The games thus only differ if $\mathcal{A}$ performed a successful C2PRI attack on KEM_0 . We turn this into a successful C2PRI reduction $\mathcal{C}$. Since C2PRI attackers on KEM_0 get the secret key used by KEM_0 and since they can draw the key pair for KEM_1 on their own, $\mathcal{C}$ can perfectly simulate the decapsulation oracle and immediately recognize and output any C2PRI solution c_0 that might appear in $\mathcal{A}$'s decapsulation queries.

$$|\Pr[G_0(\mathcal{A}) = 1] - \Pr[G_1(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_1}^{\text{C2PRI}}(\mathcal{C}) .$$

Game G_2 . We now further prevent that $\mathcal{A}$ accesses the challenge shared secret through a random oracle query: concretely, G_2 aborts whenever $\mathcal{A}$ queries the random oracle H that models H on the input that generates the challenge shared secret, i.e., $(k_1^*, k_2^*, c_2^*, \text{algolabel})$. After this change, the challenge shared secret is entirely independent of $\mathcal{A}$'s view, thus

$$\Pr[G_2(\mathcal{A}) = 1] = \frac{1}{2} .$$

Games G_1 and G_2 only differ if $\mathcal{A}$ issues this random oracle query, meaning $\mathcal{A}$ knows k_1^* and was able to break one-way security of KEM_1 . A plain one-way reduction, however, would not have the corresponding secret key. It would thus not know how to identify k_1^* without guessing, and more importantly, how to simulate the decapsulation oracle. We will thus give a OW-PCVA reduction $\mathcal{B}$ that leverages its plaintext checking oracle and its validity oracle to resolve both issues.

Before running $\mathcal{A}$, $\mathcal{B}$ picks the key pair and the encapsulation tuple (c_0^*, k_0^*) belonging to KEM_1 on its own. Whenever $\mathcal{B}$ needs to answer a fresh decapsulation query (c_0, c_1) , $\mathcal{B}$ decapsulates c_0 to obtain k_0 and uses its validity oracle to check if c_1 is valid. If it is not, $\mathcal{B}$ returns $\perp$. If it is, $\mathcal{B}$ picks a shared secret k uniformly at random and stores $((c_0, c_1), k)$ in a list to be able to repeat k on repeated query (c_0, c_1) . For simulation consistency, $\mathcal{B}$ will need to suitably simulate H —without having sk_1 , $\mathcal{B}$ needs to ensure that $\text{Decap}(sk, (c_0, c_1)) = H(k_0, \text{Decap}_1(sk_1, c_1), c_1, \text{algolabel})$ for any queried ciphertext (c_0, c_1) . To that end, $\mathcal{B}$'s simulation of the decapsulation oracle will additionally store the triplet (k_1, c_2, k) in a list $\mathcal{L}_D$. When simulating H on an input $(k_0, k_1, c_1, \text{algolabel})$, $\mathcal{B}$ uses its plaintext-checking oracle to check if c_1 decapsulates to k_1 . If this is the case, it checks if there already exists a triplet (k_0, c_1, k) in $\mathcal{L}_D$, and in that case, it returns k . Otherwise, it picks a shared secret k uniformly at random and adds the triplet (k_0, c_1, k) to the list $\mathcal{L}_D$. $\mathcal{B}$ consults $\mathcal{L}_D$ during decapsulation queries to ensure that its response is consistent with a potential previous query to H .

If $\mathcal{A}$ ever queries H on a message of the form $(k_0^*, k_1, c_1^*, \text{algolabel})$ for some k_1 , $\mathcal{B}$ uses its plaintext-checking oracle to check if c_1^* decapsulates to k_1 . It thereby correctly identifies k_1^* upon a query of $\mathcal{A}$ to H on the message $(k_0^*, k_1^*, c_1^*, \text{algolabel})$, thereby winning the OW-PCVA game.

$$|\Pr[G_1(\mathcal{A}) = 1] - \Pr[G_2(\mathcal{A}) = 1]| \leq \text{Adv}_{\text{KEM}_1}^{\text{OW-PCVA}}(\mathcal{B}).$$

□

C.2 Multi-User/Challenge IND-CCA

We argue that QSI inherently preserves multi-user/challenge IND-CCA security for natural base KEMs. The multi-user/challenge IND-CCA game captures n_u many honest users by sampling n_u many key pairs. It captures that they can be attacked multiple times by providing a challenge oracle Chal that can be called n_c many times. Chal can be called on any user index $i \in [n_u]$. For the j -th call $\text{Chal}(i)$, it will then generate an IND-CCA challenge $(c_j, K_{j,b})$ according to the i -th key pair and the game's challenge bit b . As in the plain IND-CCA game, the attacker is tasked with determining b .

Next, we give a high-level sketch of how the plain IND-CCA proof could be adapted to the multi-user/challenge setting. Here, we'll only discuss the case considering C2PRI-security of KEM_0 and IND-CCA-security of KEM_1 . Since the reverse case is completely symmetrical, this is without loss of generality.

Proof (Sketch). We briefly recall the proof of the “plain IND-CCA” Theorem 14: to argue that the honest key $k^* = H(k_0^*, k_1^*, \text{algolabel})$ is (computationally) indistinguishable from random, the proof

1. ensured that k^* will never be given out as a permitted decapsulation, based on C2PRI security of KEM_0 ,
2. used IND-CCA security of KEM_1 to argue that k_1^* can be replaced with random,
3. employed the skPRF property of H to replace the challenge shared secret k^* with random,
4. and undid the change to k_1^* and the C2PRI-related abort.

To capture that n_u many honest users that can be attacked n_c many times, we can start by adapting step 1; to prevent that any of the n_c many challenge secrets $k^1, \dots, k^{n_c}$ are given out as a permitted decapsulation, by means of an abort whenever this would happen. To that end, one can use a multi-user variant of C2PRI for KEM_0 , in which the attacker is given n_u many key pairs $(pk_1, sk_1), \dots, (pk_{n_u}, sk_{n_u})$ sampled according to KeyGen_1 , can adaptively select key pairs out of those for which it can then request a challenge ciphertext sampled according to $(c_j, k_j) \leftarrow \text{Encap}_0(pk_i)$, and wins if it outputs a user index i , a challenge index j that happened using pk_i , and a second preimage, so a ciphertext $c'_j \neq c_j$ such that $\text{Decap}_0(sk_i, c'_j) = k_j$.

To adapt step 2, we can use multi-user IND-CCA security of KEM_1 to replace the KEM_1 -related shared secrets within the challenges with random. The respective reduction can sample the key pairs for KEM_0 on its own and will simulate the QSI challenge oracle by sampling honest encapsulations for KEM_0 and calling its own challenge oracle for KEM_1 . It can simulate the decapsulation oracles for the n_u many users by using the key pairs for KEM_0 and the respective decapsulation oracles for KEM_1 . When the attacker queries the decapsulation oracle on a ciphertext (c_0, c_1) that has been sampled as one of the QSI challenges for the same user, the reduction will use the corresponding KEM_1 challenge to derive its response. Similar to the plain IND-CCA proof, this simulation works unless one of the KEM_1 challenge ciphertexts fails to decapsulate. This happens with probability at most δ'_1 , when δ'_1 is an upper bound for any adversary's success advantage in the following adaptive correctness game: the game prepares n_u many key pairs, and the adversary is allowed to trigger

Encap n_c many times, choosing which of the sampled key pairs is being used each time, and wins if any of the sampled decapsulations triggers decryption failure.

To adapt step 3, we can use a multi-user variant of the skPRF property of H twice to replace the n_c many challenge secrets $k^1, \dots, k^{n_c}$ with random. In that variant, the attacker would still have adaptive control over all skPRF key components $k_1, \dots, k_{j-1}, k_j, k_{j+1}, \dots, k_n$ except for the component slot for k_j , for which the game samples n_c many, and the attacker gets access to an oracle that looks like the plain skPRF oracle, except that the attacker uses an additional input i to specify which of the n_c many samples is supposed to be used in the j -th key component slot.

To adapt step 4, we can then once more use multi-user IND-CCA security of KEM_1 to switch the KEM_1 -related shared secrets within the challenges back to honest, and lastly, lift the abort, by invoking the multi-user variant of C2PRI for KEM_0 once more. $\square$

D Binding: Remaining Definitions and Security Proofs

D.1 Definition: Malicious Binding (MAL)

The main body of the paper covers the definitions of all binding properties in the HON (honest) and LEAK (leaking) adversarial models. This leaves the strongest model in MAL (malicious), which captures a scenario where the adversary is able to determine the considered key material itself (instead of these being honestly generated). Evidently, this is a rather strong adversarial model. In fact, it is so strong that, without additional restrictions, it allows the adversary to trivially break any binding property with the public key as binding target by providing key pairs with unrelated secret and public keys. To remedy this, we follow the approach of Schmieg in [Sch24] and introduce an additional function $\text{sktopk} : \mathcal{SK} \rightarrow \mathcal{PK}$ that (honestly) computes a public key corresponding to a secret key, in line with the key generation procedure of the considered KEM.²³ Then, rather than entire *key pairs*, the adversary provides secret decapsulation keys only, and the corresponding public keys are computed through sktopk . Beyond supplying decapsulation keys, the adversary may choose to provide randomness instead of one or both encapsulations. The encapsulations are then produced through the KEM's encapsulation procedure, using the provided randomness as input. The formal definition is given below.

Definition 40 (KEM Binding (MAL) [CDM24]). Let $P, Q \subset \{\text{PK}, \mathcal{C}, \mathcal{K}\}$ such that $P \cap Q = \emptyset$, where $\text{PK} := \{\text{pk}_0, \text{pk}_1\}$, $\mathcal{C} := \{c_0, c_1\}$, and $\mathcal{K} := \{k_0, k_1\}$. Further, define “binding break” predicate

$$\begin{aligned} \text{BB}_{P,Q} : \mathcal{PK} \times \mathcal{PK} \times \mathcal{C} \times \mathcal{C} \times \mathcal{K} \times \mathcal{K} &\rightarrow \{\text{true}, \text{false}\} \\ \text{BB}_{P,Q}(\text{pk}_0, \text{pk}_1, c_0, c_1, k_0, k_1) &:= k_0 \neq \perp \wedge k_1 \neq \perp \wedge (\forall p \in P : p_0 = p_1) \wedge (\exists q \in Q : q_0 \neq q_1) , \end{aligned}$$

where p_b (resp. q_b) refers to the respective element in the set p (resp. q); e.g., if $p = \text{PK}$, p_0 denotes pk_0 . Lastly, let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be a KEM for which the function $\text{sktopk} : \mathcal{SK} \rightarrow \mathcal{PK}$ maps secret keys to public keys in line with KeyGen . Then, given an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ playing in the MAL-BIND-P-Q game (as defined in Fig. 15), the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{KEM}}^{\text{MAL-BIND-P-Q}}(\mathcal{A}) := \Pr[\text{MAL-BIND-P-Q}_{\text{KEM}}(\mathcal{A}) = 1] .$$

²³ For many KEMs, among which most of the widely considered post-quantum ones, this is trivial since the secret key typically contains the entire public key or some information to recompute it.

Game MAL-BIND-P-Q_{KEM}($\mathcal{A}$): <pre> 01 $d \leftarrow \mathcal{A}_1()$ 02 if $d = \text{DecapDecap}$ 03 $\text{sk}_0, \text{sk}_1, c_0, c_1 \leftarrow \mathcal{A}_2()$ 04 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 05 $k_0 \leftarrow \text{Decap}(\text{sk}_0, c_0)$ 06 $k_1 \leftarrow \text{Decap}(\text{sk}_1, c_1)$ 07 else if $d = \text{EncapDecap}$ 08 $\text{sk}_0, \text{sk}_1, r_0, c_1 \leftarrow \mathcal{A}_2()$ 09 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 10 $k_0, c_0 \leftarrow \text{Encap}(\text{pk}_0; r_0)$ 11 $k_1 \leftarrow \text{Decap}(\text{sk}_1, c_1)$ 12 else // $d = \text{EncapEncap}$ 13 $\text{sk}_0, \text{sk}_1, r_0, r_1 \leftarrow \mathcal{A}_2()$ 14 $\text{pk}_0, \text{pk}_1 \leftarrow \text{sktopk}(\text{sk}_0), \text{sktopk}(\text{sk}_1)$ 15 $k_0, c_0 \leftarrow \text{Encap}(\text{pk}_0; r_0)$ 16 $k_1, c_1 \leftarrow \text{Encap}(\text{pk}_1; r_1)$ 17 return $\text{BB}_{P,Q}(\text{pk}_0, \text{pk}_1, c_0, c_1, k_0, k_1)$ </pre>

Fig. 15: MAL-BIND-P-Q game for $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ and $P, Q \subset \{\text{PK}, \text{C}, \text{K}\}$ such that $P \cap Q = \emptyset$.

D.2 Honest Binding (HON) for QSF & QSI

The binding properties in the HON and LEAK model (see Definition 9) share a similar structure, with a single difference: In the HON model, the adversary has access to a decapsulation oracle; in the LEAK model, the adversary has access to the decapsulation keys. Importantly, in both models, all key material is honestly generated and remains beyond the adversary’s control. This similarity enables both the results and proofs for QSF and QSI to almost directly carry over between the models, requiring only minor adjustments to account for the difference in adversarial capabilities and environment. Notably, these adjustments necessitate little novel reasoning or insights, allowing for a straightforward translation; furthermore, they are nearly identical for all properties and even across both combiners. As such, to avoid unnecessary repetition, we present only one full proof (for QSI) in the HON model, highlighting the similarities and differences with their counterparts in the LEAK model. For the remaining results, the proofs mostly refer back to the relevant steps in other proofs.

QSI. The results we establish for QSI in the HON model are direct analogs of those in the LEAK model. Particularly, this means we consider the same categorization based on (1) the use of a (collision-resistant) hash function, and (2) the kind of preservation (existential or universal). Furthermore, the results hold in both in the classical and quantum setting, though with slightly varying concrete bounds.

Remark 41. As in the leaking model, the results in the honest model that involve a CR advantage term can be made slightly more concrete by modeling the employed hash function as a random oracle and replacing the advantage term by the corresponding collision bound.

In contrast to the leaking model, however, the reduction adversary in the honest model must additionally simulate the decapsulation oracle. This simulation requires querying the random oracle to derive the final shared key for each decapsulation query. As a consequence, the total number of random-oracle queries made by the reduction adversary equals the sum of the random-oracle

queries and decapsulation-oracle queries made by the adversary against the binding property of the combiner, plus two additional queries for the final decapsulations.

From the six properties, we provide a detailed proof for HON-BIND-K-CT (and HON-BIND-K,PK-CT), as it incorporates all the necessary modifications for transitioning results and proofs from the LEAK model. As such, it offers an instructive example and serves as a useful reference for other proofs.

Theorem 42 (QSI Universal Preservation of HON-BIND-K-CT and HON-BIND-K,PK-CT). *Let KEM_0 and KEM_1 be KEMs with respective key spaces $\mathcal{K}_0$ and $\mathcal{K}_1$, $\text{algotlabel} \in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{K}_1 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSI}[\text{KEM}_0, \text{KEM}_1, H]$. Then, for any adversary $\mathcal{A}$ against HON-BIND-P-Q $\in \{\text{HON-BIND-K-CT}, \text{HON-BIND-K,PK-CT}\}$ for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_H^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{HON-BIND-P-Q}}(\mathcal{C}) + \text{Adv}_{\text{KEM}_1}^{\text{HON-BIND-P-Q}}(\mathcal{D}) .$$

In the above, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$, $\mathcal{C} := \mathcal{C}(\mathcal{A})$, and $\mathcal{D} := \mathcal{D}(\mathcal{A})$ are as constructed in the proof of this theorem, with each of $\mathcal{C}$ and $\mathcal{D}$ issuing at most q_{DO} queries to its decapsulation oracle.

Proof. Foremost, the conditions for success in HON-BIND-K-CT are identical to those for the analogous property in the LEAK model, as the $\text{BB}_{\text{P,Q}}$ predicate reduces to the same formula: $k_0 = k_1 \neq \perp \wedge c_0 \neq c_1$. To reiterate, the adversary must provide two *different* encapsulations $c_0 := (c_{00}, c_{01}) \neq (c_{10}, c_{11}) =: c_1$ that decapsulate to the *same* shared keys $k_0 = k_1$ under decapsulation keys $\text{sk}_0 := (\text{sk}_{00}, \text{sk}_{01})$ and $\text{sk}_1 := (\text{sk}_{10}, \text{sk}_{11})$ that *may or may not* be equal (as $\mathcal{A}$ may decide itself whether to consider the same or independently generated key pairs). Following the reasoning from the proof for LEAK-BIND-K-CT (Theorem 16), this means that all corresponding inputs to H (computing the final shared keys) are equal *unless* a collision occurred. Thus, as before, splitting the probability space based on an event D that captures the case where *any* of the corresponding inputs are unequal, we can construct an adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ such that

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-K-CT}}(\mathcal{A}) \leq \Pr[\text{HON-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D] + \text{Adv}_H^{\text{CR}}(\mathcal{B}) ,$$

At this point, we can continue exactly as in the proof for LEAK-BIND-K-CT, merely adjusting the reduction adversaries to simulate the decapsulation oracle for $\mathcal{A}$. That is, we split the probability space on the event $C := c_{00} \neq c_{10}$ (at this point, $\neg C$ implies $c_{01} \neq c_{11}$). Then, in the case where C holds, we simultaneously have $c_{00} \neq c_{10}$ and $\text{KEM}_0.\text{Decap}(c_{00}, \text{sk}_0) = \text{KEM}_0.\text{Decap}(c_{10}, \text{sk}_0)$. Hence, we can construct an adversary $\mathcal{C} := \mathcal{C}(\mathcal{A})$ against HON-BIND-K-CT for KEM_0 that, first, runs $\mathcal{A}_1$ and directly returns its output, indicating whether to consider the same or independently generated key pairs. Upon receiving the (potentially duplicated) key pairs from its own game, $\mathcal{C}$ behaves similarly to the analogous reduction adversary in the proof for LEAK-BIND-K-CT—i.e., it generates key pair(s) with KEM_1 , combines them with the ones (for KEM_0) received from its own game,²⁴ and runs $\mathcal{A}_2$ on the result while perfectly simulating any oracle calls—but additionally simulates the decapsulation oracle. Specifically, it does so by performing (1) the KEM_0 decapsulations using its own decapsulation oracle, (2) the KEM_1 decapsulations using the secret key(s) it generated itself earlier, and (3) the derivation of the final shared key using H . After $\mathcal{A}_2$ terminates, $\mathcal{C}$ extracts and returns the break for its own game, contained in $\mathcal{A}_2$'s output; hence, we establish the following bound:

$$\Pr[\text{HON-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge C] \leq \Pr[\text{HON-BIND-K-CT}_{\text{KEM}_0}(\mathcal{C}) = 1] .$$

²⁴ Naturally, in this case, the reduction adversary only combines and passes the *public keys*, as this is what $\mathcal{A}$ expects (and the reduction adversary does not get the secret keys from its own game). This is different from (the proof for) LEAK-BIND-K-CT, which deals with entire key pairs.

The other case (where $\neg C$ holds), is completely symmetrical. That is, we can similarly construct an adversary $\mathcal{D} := \mathcal{D}(\mathcal{A})$ —obtained by adapting its counterpart in the proof for LEAK-BIND-K-CT in an analogous manner (to $\mathcal{C}$)—such that

$$\Pr[\text{HON-BIND-K-CT}_{\text{KEM}}(\mathcal{A}) = 1 \wedge \neg D \wedge \neg C] \leq \Pr[\text{HON-BIND-K-CT}_{\text{KEM}_1}(\mathcal{D}) = 1] .$$

The desired bound follows by combining the individual steps.

For HON-BIND-K,PK-CT, a simplified version of the proof applies, obtained by adjusting the reduction adversaries from the corresponding proof in the LEAK model in the same manner as above. $\square$

Theorem 43 (QSI Existential Preservation of HON-BIND-K-PK and HON-BIND-K,CT-PK). *Consider the same setting as for Theorem 42. Then, for any adversary $\mathcal{A}$ against $\text{HON-BIND-P-Q} \in \{\text{HON-BIND-K-PK}, \text{HON-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_b}^{\text{HON-BIND-P-Q}}(\mathcal{C}_b) + \epsilon_b ,$$

for $b \in \{0, 1\}$. In the above, $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C}_b := \mathcal{C}_b(\mathcal{A})$ are as constructed in the proof of this theorem—with $\mathcal{C}_b$ issuing at most q_{DO} queries to its decapsulation oracle—and ϵ_b denotes the probability of KEM_b generating the same public key in two independent executions of its key generation procedure.

Proof. The proofs proceed in the same manner as those for LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK (Theorem 17), appropriately adjusted to account for the differences in adversarial capabilities and environment (i.e., in terms of oracles and input/output values). These adjustments are analogous to the ones in Theorem 42. $\square$

Theorem 44 (QSI Universal Preservation of HON-BIND-CT-K). *Consider the same setting as for Theorem 42, but let H be any function. Then, for any adversary $\mathcal{A}$ against HON-BIND-CT-K for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-CT-K}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{HON-BIND-CT-K}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_1}^{\text{HON-BIND-CT-K}}(\mathcal{C}) .$$

Here, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem, with each issuing at most q_{DO} queries to its decapsulation oracle.

Proof. The proof is similar to that for LEAK-BIND-CT-K (Theorem 18), appropriately adjusted to account for the differences in adversarial capabilities and environment. These adjustments are analogous to the ones in Theorem 42. $\square$

Theorem 45 (QSI Existential Preservation of HON-BIND-CT-PK). *Consider the same setting as for Theorem 42, but let H be any function. Then, for any adversary $\mathcal{A}$ against HON-BIND-CT-PK for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_b}^{\text{HON-BIND-CT-PK}}(\mathcal{B}_b) + \epsilon_b ,$$

for $b \in \{0, 1\}$. Here, $\mathcal{B}_b := \mathcal{B}_b(\mathcal{A})$ is as constructed in the proof of this theorem—issuing at most q_{DO} queries to its decapsulation oracle—and ϵ_b denotes the probability of KEM_b generating the same public key in two independent executions of its key generation procedure.

Proof. The proof is similar to that for LEAK-BIND-CT-PK (Theorem 19), appropriately adjusted to account for the differences in adversarial capabilities and environment. These adjustments are analogous to the ones in Theorem 42. $\square$

QSF. As for QSI, the results in the HON model for QSF are direct analogs of those in the LEAK model. Moreover, the adaptations required for each of the proofs are similar to those needed for (the proofs of the same property of) QSI. Certainly, since the results we established for QSF only rely on (non-interactive) statistical properties as far as the nominal group is concerned, the proofs require no changes related to the reasoning around the nominal group. Hence, nearly all required changes concern the reasoning around the oracles, which is identical to that for QSI. For this reason, we do not cover any proof in detail, but back-reference the appropriate proof steps instead.

Theorem 46 (QSF Preservation of HON-BIND-K-CT and HON-BIND-K,PK-CT). *Let KEM_0 be a KEM with key space $\mathcal{K}_0$, $\mathcal{N}$ a nominal group, $\text{algotlabel} \in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{G}^3 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSF}[\text{KEM}_0, \mathcal{N}, H]$. Then, for any adversary $\mathcal{A}$ against $\text{HON-BIND-P-Q} \in \{\text{HON-BIND-K-CT}, \text{HON-BIND-K,PK-CT}\}$ for KEM (as defined in Fig. 4), issuing $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{HON-BIND-P-Q}}(\mathcal{C}) .$$

In the above, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem, with $\mathcal{C} := \mathcal{C}(\mathcal{A})$ issuing at most q_{DO} queries to its decapsulation oracle.

Proof. The proofs are similar to that for LEAK-BIND-K-CT and LEAK-BIND-K,PK-CT (Theorem 20), appropriately adjusted to account for the differences in adversarial capabilities and environment. These adjustments are analogous to the ones in Theorem 42. $\square$

Theorem 47 (QSF HON-BIND-K-PK and HON-BIND-K,CT-PK). *Consider the same setting as for Theorem 46. Then, for any adversary $\mathcal{A}$ against $\text{HON-BIND-P-Q} \in \{\text{HON-BIND-K-PK}, \text{HON-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{CR}}(\mathcal{B}) + \epsilon_{\mathcal{N}} .$$

Here, $\mathcal{B} := \mathcal{B}(\mathcal{A})$ is as constructed in the proof of this theorem, and $\epsilon_{\mathcal{N}}$ denotes the probability of $\mathcal{N}$ generating the same public key in two independent executions of its key generation procedure.

Proof. The proofs are similar to that for LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK (Theorem 21), appropriately adjusted to account for the differences in adversarial capabilities and environment. These adjustments are analogous to the ones in Theorem 42. $\square$

Theorem 48 (QSF Preservation of HON-BIND-CT-PK). *Consider the same setting as for Theorem 46, but let H be any function. Then, for any adversary $\mathcal{A}$ against HON-BIND-CT-PK for KEM (as defined in Fig. 4), issuing at most $q_{\text{DO}} \geq 0$ queries to DO_{Decap} , it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{HON-BIND-CT-PK}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{HON-BIND-CT-PK}}(\mathcal{B}) + \epsilon_0 .$$

Here, adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ is as constructed in the proof of this theorem—issuing at most q_{DO} queries to its decapsulation oracle—and ϵ_0 denotes the probability of KEM_0 generating the same public key in two independent executions of its key generation procedure.

Proof. The proof is similar to that for LEAK-BIND-CT-PK (Theorem 22), appropriately adjusted to account for the differences in adversarial capabilities and environment. These adjustments are analogous to the ones in Theorem 42. $\square$

Lastly, QSF does *not* (necessarily) preserve HON-BIND-CT-K for similar reasons it does not do so in the LEAK model analog.

D.3 Malicious Binding (MAL) for QSF & QSI

In the MAL model, the situation differs from that of the other models. This is primarily due to the adversary’s ability to generate key material, which invalidate arguments that rely on the probability of independently generated public keys being distinct. Such reasoning played a role in the proofs of several results for the HON and LEAK models, and was the main differentiating factor between the results. However, the other core techniques—namely, arguments concerning (1) collisions in the hash function, and (2) the extraction of an analogous break for a constituent KEM from a break against the combiner—remain applicable. These techniques can be lifted and extended to the MAL model to still establish preservation results. Nevertheless, with the main source of differentiation removed, the results we establish can essentially be unified in a single category for each of the combiners.

QSI. For QSI, the results we establish in the MAL model are all instances of universal preservation. That is, if *both* of the constituent KEMs satisfy a certain binding property, QSI does as well. As in the other models, the hash function is required to be collision resistant for properties where the shared key is a binding source. To keep the statements clear and concise, we separate the results for QSI into two theorems. Although both can be categorized as universal preservation, we split them into those that have the shared key as binding source and those that do not.

Theorem 49 (QSI Universal Preservation of Malicious Binding (K as Source)). *Let KEM_0 and KEM_1 be KEMs with respective key spaces $\mathcal{K}_0$ and $\mathcal{K}_1$, $\text{algotlabel} \in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{K}_1 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSI}[\text{KEM}_0, \text{KEM}_1, H]$. Then, for any adversary $\mathcal{A}$ against $\text{MAL-BIND-P-Q} \in \{\text{MAL-BIND-K-CT}, \text{MAL-BIND-K,PK-CT}, \text{MAL-BIND-K-PK}, \text{MAL-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 15), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{MAL-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_H^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{MAL-BIND-P-Q}}(\mathcal{C}) + \text{Adv}_{\text{KEM}_1}^{\text{MAL-BIND-P-Q}}(\mathcal{D}) + \text{Adv}_{\text{KEM}_1}^{\text{MAL-BIND-P-Q}}(\mathcal{C}) .$$

In the above, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$, $\mathcal{C} := \mathcal{C}(\mathcal{A})$, and $\mathcal{D} := \mathcal{D}(\mathcal{A})$ are as constructed in the proof of this theorem.

Proof. For all properties, the proof proceeds similarly. First, akin to the proofs for the analogous properties in the other models, we commence by noting that the $\text{BB}_{P,Q}$ predicate reduces to one that requires the adversary to return values such that, in the end, the shared keys are the same. Per the decapsulation procedure of KEM, this means that the key derivation performed by the two decapsulations gives the same output. In turn, we can carve out the case where any of the corresponding inputs are unequal, implying a collision for H occurred, and bound accordingly. This gives rise to the CR advantage term for $\mathcal{B} := \mathcal{B}(\mathcal{A})$.

Then, it remains to bound the case where all corresponding inputs to H are equal, at which point we can deduce that the break returned by $\mathcal{A}$ *must* contain an analogous break one of the constituent KEMs:

- **MAL-BIND-K-CT.** The encapsulations (c_{00}, c_{01}) and (c_{10}, c_{11}) returned by $\mathcal{A}$ (either directly or indirectly by providing the randomness used to create it/them) are unequal. In particular, this means that $c_{00} \neq c_{10} \vee c_{01} \neq c_{11}$. However, per the considered case, we have that both $\text{KEM}_0.\text{Decap}(\text{sk}_{00}, c_{00}) = \text{KEM}_0.\text{Decap}(\text{sk}_{10}, c_{10})$ and $\text{KEM}_0.\text{Decap}(\text{sk}_{01}, c_{01}) = \text{KEM}_0.\text{Decap}(\text{sk}_{11}, c_{11})$. As such, if $c_{0b} \neq c_{1b}$, then the break for KEM contains a break for KEM_b .
- **MAL-BIND-K,PK-CT.** As MAL-BIND-K-CT, but with equal public keys. Notice that, equal public keys for KEM implies equal public keys for both constituent KEMs. So, whichever part of the encapsulations is unequal, the public keys for the corresponding constituent KEM are always equal (which is the additional requirement for the break to be valid in this case).
- **MAL-BIND-K-PK.** Analogous to MAL-BIND-K-CT, but applying the reasoning around the encapsulations to the public keys instead.

- MAL-BIND-K,CT-PK. As MAL-BIND-K,PK-CT, but interchanging the roles of (and reasoning for) the encapsulations and public keys.

Thus, as long as we can construct reduction adversaries that properly simulate $\mathcal{A}$'s environment, we can bound the remaining cases by reducing from the corresponding binding properties of the constituent KEMs. However, since $\mathcal{A}$'s entire environment consists only of H (i.e., no other inputs or oracles),²⁵ this simulations is trivial and, moreover, identical across all considered properties. That is, the reduction adversaries (1) run $\mathcal{A}_1$, directly forwarding any of its queries to H , and immediately return its output; and (2) run $\mathcal{A}_2$, again directly forwarding any of its queries to H , and straightforwardly extract and return the values relevant for their own game from its output. As an example, consider a reduction adversary against KEM_b (independent of the specific property). If $\mathcal{A}_1$ returns $d = \text{EncapsDecaps}$, then $\mathcal{A}_2$ returns two secret decapsulation keys, randomness, and an encapsulation. All of these are simply composed of a part for KEM_0 and a part for KEM_1 . Therefore, the reduction adversary can simply extract and return the values for KEM_b . (Since the reduction adversary also directly returned $d = \text{EncapsDecaps}$, this matches the values it is expected to provide.) Comparable reasoning applies when $\mathcal{A}_1$ chooses d to be DecapsDecaps or EncapsEncaps . By the considered case, the reduction adversary succeeds in its own game and we can bound accordingly. This leads to the advantage terms in the bound. Combining the individual steps yields the desired result. $\square$

Theorem 50 (QSI Universal Preservation of Malicious Binding (K Not as Source)). *Consider the same setting as for Theorem 49, but let H be any function. Then, for any adversary $\mathcal{A}$ against $\text{MAL-BIND-P-Q} \in \{\text{MAL-BIND-CT-K}, \text{MAL-BIND-CT-PK}\}$ for KEM (as defined in Fig. 15), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{MAL-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_0}^{\text{MAL-BIND-P-Q}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_1}^{\text{MAL-BIND-P-Q}}(\mathcal{C}) .$$

Here, adversaries $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ are as constructed in the proof of this theorem.

Proof. Once again, for both proofs, the first reasoning steps match those in the proofs for the analogous properties in the other models (because the final $\text{BB}_{P,Q}$ predicate is independent of the model). Subsequently, the construction of the reduction adversaries closely follows that in the proof for Theorem 49.

For MAL-BIND-CT-K, we start by noting that, since the final shared keys are unequal, at least one of the corresponding inputs to the key derivation function must be as well. Here, we distinguish two cases: the shared keys produced by KEM_0 are unequal, or they are not, in which case the shared keys produced by KEM_1 must be unequal. In the former, the break returned by $\mathcal{A}$ contains an analogous break for KEM_0 : the corresponding encapsulations are equal by the assumption that $\mathcal{A}$ succeeds, and the corresponding shared keys are unequal by the considered case. The latter case is entirely symmetrical. So, in either case, we can construct a reduction adversary that perfectly simulates $\mathcal{A}$'s environment—akin to the ones in the proof for Theorem 49—extracts the relevant break, and succeeds in its own game. The desired result follows.

For MAL-BIND-CT-PK, there is no requirement on the shared keys at all, except for them not being equal to $\perp$, i.e., the corresponding decapsulations must succeed. Per the decapsulation procedure of KEM, this means that the final key derivation went through and, particularly, that the procedure was *not* aborted due to a failure in the decapsulations by KEM_0 and KEM_1 . By the assumption that $\mathcal{A}$ succeeds, the overall KEM public key must be different. Since this simply consists of a part for KEM_0 and a part for KEM_1 , we can distinguish two cases: the public keys for KEM_0 are unequal, or they are not, in which case the public keys for KEM_1 must be unequal. Since the encapsulations

²⁵ Also, there is no need for any advanced techniques with respect to the random oracle.

for both constituent KEMs are equal (again by the assumption that $\mathcal{A}$ succeeds), the former (resp. latter) case encompasses a MAL-BIND-CT-PK break for KEM_0 (resp. KEM_1). In either case, we can construct a reduction adversary that perfectly simulates the environment of $\mathcal{A}$ —similar to the ones in the proof for Theorem 49—extracts the relevant break, and succeeds in its own game. The desired result follows. $\square$

QSF. As for QSI, essentially all results we establish for QSF in the MAL model fall under the same category. Specifically, this category concerns “regular” preservation: QSF preserves the binding property satisfied by its (only) constituent KEM. Indeed, because QSF uses all artifacts from its nominal group as inputs to the final key derivation, it does not need to depend on any binding-like properties of this group for properties with the shared key as binding source, even in the strongest model. (Provided the key derivation function is collision resistant.)

Interestingly, however, QSF does *not* satisfy any property without the key as binding source—i.e., it does not satisfy MAL-BIND-CT-K and MAL-BIND-CT-PK. Concretely, the adversary may behave as follows to break the property. First, $\mathcal{A}_1$ chooses any d , which one is somewhat irrelevant; for simplicity, say it chooses DecapsDecaps. Second, $\mathcal{A}_2$ generates a (single) honest key pair for KEM_0 and uses it to produce a honest encapsulation. Moreover, $\mathcal{A}_2$ generates two (honest) key pairs for the nominal group—making sure they are different—and creates an encapsulation by picking another (honest) secret exponent and raising the base element to it. Recall that, due to $\mathcal{A}_1$ ’s choice of DecapsDecaps, $\mathcal{A}_2$ must return two secret decapsulation keys and two encapsulations, each consisting of a part for KEM_0 and a part for $\mathcal{N}$. In addition, to win the game, (1) the encapsulations must be equal and (2) the public keys (obtained from the provided secret keys) must be different. So, for the two KEM_0 secret key parts, $\mathcal{A}_2$ uses the single such secret key it generated, twice; for the two KEM_0 encapsulation parts, it uses the single such encapsulation it produced, twice. Further, for the two $\mathcal{N}$ secret key parts, it uses both secret exponents it picked, once each; for the two $\mathcal{N}$ encapsulation parts, it uses the single such encapsulation it produced, twice. Crucially, both the resulting secret keys and encapsulations contain *identical and honest* KEM_0 parts, meaning the subsequent corresponding decapsulations performed by the game will succeed (the fact it produces the same result is irrelevant) and, particularly, are *not* aborted. Now, by construction, the $\mathcal{N}$ parts of the secret keys are different secret exponents that correspond to different public keys, implying the overall public keys (for QSF) are different as well, as needed for the adversary to succeed. Furthermore, the $\mathcal{N}$ parts of the encapsulations are equal, implying the overall encapsulations (for QSF) are equal as well, satisfying the other requirement for the adversary to succeed. (Here, the “decapsulation” operation can not fail, so we do not have to consider this possibility.) Concluding, the adversary ends up providing different public keys with the same encapsulation such that all decapsulations succeed. Indeed, this breaks MAL-BIND-CT-PK for QSF.²⁶

At this point, it remains to go over the binding properties that *do* have the shared key as binding source. For these properties, we consider one combined statement and proof, given below.

Theorem 51 (QSF Preservation of Malicious Binding (K as Source)). *Let KEM_0 be a KEM with key space $\mathcal{K}_0$, $\mathcal{N}$ a nominal group, $\text{algotlabel} \in \mathcal{L}$ an algorithm-specific label, and $H : \mathcal{K}_0 \times \mathcal{G}^3 \times \mathcal{L} \rightarrow \mathcal{K}$ a hash function. Further, define $\text{KEM} := \text{QSF}[\text{KEM}_0, \mathcal{N}, H]$. Then, for any adversary $\mathcal{A}$ against $\text{MAL-BIND-P-Q} \in \{\text{MAL-BIND-K-CT}, \text{MAL-BIND-K,PK-CT}, \text{MAL-BIND-K-PK}, \text{MAL-BIND-K,CT-PK}\}$ for KEM (as defined in Fig. 15), it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{MAL-BIND-P-Q}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{H}}^{\text{CR}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_0}^{\text{MAL-BIND-P-Q}}(\mathcal{C}) .$$

In the above, adversary $\mathcal{B} := \mathcal{B}(\mathcal{A})$ and $\mathcal{C} := \mathcal{C}(\mathcal{A})$ is as constructed in the proof of this theorem.

²⁶ Notice that it does not matter whether the resulting shared keys are equal or not; the only requirement related to the shared keys is that they are not $\perp$, i.e., that the decapsulations succeed.

Proof. Here, the analysis is essentially a one-sided version of that for Theorem 49; or, put differently, a combination of the analyses for Theorem 49 and those for the analogous properties in the LEAK model (Theorems 20 and 21). In short, for each of these properties, we start by excluding and bounding the case of a collision in H , leading to the CR advantage term. Subsequently, in the remaining case, all inputs to the random oracle are equal; particularly, the shared keys, public keys, and encapsulations originating from the nominal group are equal. It follows that, in this case, the break returned by $\mathcal{A}$ must contain an analogous break for KEM_0 . As such, we can construct a reduction adversary in a similar fashion as we did in Theorem 49 that perfectly simulates $\mathcal{A}$'s environment and extracts the break, succeeding in its own game. From this, the desired results follows. $\square$

Remark 52. As before, the results in the malicious model that involve a CR advantage term can be made concrete by replacing this term with the corresponding collision bound in the random-oracle model. The number of random-oracle queries made by the reduction adversary is identical to that in the leaking model, and the resulting collision bound is exactly the same.